REPUBLIQUE DU CAMEROUN
PAIX - TRAVAIL – PATRIE
****
UNIVERSITE DE DSCHANG
****
ÉCOLE DOCTORALE



REPUBLIC OF CAMEROON
PEACE - WORK - FATHERLAND
****
UNIVERSITY OF DSCHANG
****
POST GRADUATE SCHOOL

DSCHANG SCHOOL OF SCIENCES AND TECHNOLOGY

UNITE DE RECHERCHE EN INFORMATIQUE FONDAMENTALE, INGENIERIE ET APPLICATIONS (URIFIA)

SUJET :

VERS UNE APPROCHE BASÉE SUR L'INGÉNIERIE DIRIGÉE PAR LES MODÈLES, DE GESTION DES WORKFLOWS DANS UNE APPLICATION WEB

Mémoire soutenu publiquement en vue de l'obtention du
diplôme de Master en Informatique

Option : Informatique Fondamentale
Spécialité : Intelligence Artificielle

Par

ZEMTSOP NDADJI Brice Arléon
*Matricule : **CM-UDS-17SCI2554***
Licence en Informatique Fondamentale

Sous la direction de :

TCHOUPÉ TCHENDJI Maurice
Maître de conferences, Université de Dschang, Cameroun

Année académique 2021/2022

DÉDICACES

À mon père NDADJI EMMANUEL *et à ma mère* MAFFO ZEMTSOP MARIE.

REMERCIEMENTS

Mes remerciements les plus sincères :

— À Dieu Tout-Puissant sans qui rien n'aurait été possible ;
— Au Pr TCHOUPE TCHENDJI Maurice pour la confiance qu'il m'a témoigné en acceptant la direction scientifique de mes travaux. Je lui suis reconnaissant de m'avoir fait bénéficier tout au long de ce travail, de sa rigueur intellectuelle, de sa disponibilité, de sa bienveillance, de son efficacité ;
— Au Dr ZEKENG NDADJI Milliam Maxime, mon grand frère et mon mentor depuis le début de mon parcours universitaire. Les mots ne seront jamais assez pour lui témoigner ma gratitude, ma reconnaissance et surtout mon profond respect ;
— À tous les membres du jury, pour l'immense honneur que vous m'accordez en appréciant ce travail ;
— À tous les enseignants du département de Mathématiques-Informatique pour leur aide et leurs encouragements constants ;
— À mes chers parents, NDADJI Emmanuel et MAFFO ZEMTSOP Marie pour la confiance qu'ils ont placé en moi, leur amour et leurs bénédictions.
— À mes frères et sœurs (Chanceline, Maxime, Emanie, Alex, Arnold) ;
— À mes amis (Daniela, Brell, Nina, Wilfried, Baudouin, Flore, Franck, Ismael, Eloyise, Irène, Benedicte, Chérone, Yohanie ...) ;
— À tous mes camarades de promotion de Master 2 pour les discussions constructives que nous avons partagées ;
— À la structure Dimsoft/CeNKYAH pour m'avoir offert de meilleures conditions de travail et aussi pour les compétences que j'ai pu y acquérir ;
— À tous ceux qui m'ont soutenu et dont j'ai omis de mentionner les noms.

TABLE DES MATIÈRES

Dédicaces i
Remerciements ii
Table des matières iii
Liste de symboles vii
Liste des acronymes viii
Liste des tableaux ix
Table des figures x
Résumé xi
Abstract xii

Introduction Générale **1**
Contexte du travail 1
Problématique et objectifs 2
Organisation du manuscrit 3

Chapitre I ▶ **Généralités sur le Business Process Management (BPM) et le Développement web** **5**
I.1 - Introduction 5
I.2 - Généralités sur le Business Process Management (BPM) 6
I.2.1 - Définitions et Historique 7
I.2.1.1 - Définitions 7
I.2.1.2 - Historique du BPM 9
I.2.2 - Cycle de vie du BPM 10
I.2.2.1 - true 10
I.2.2.2 - Découverte des processus (Process discovery) 11
I.2.2.3 - Analyse du processus (Process analysis) 11
I.2.2.4 - Refonte du processus (Process redesign) 11
I.2.2.5 - Mise en œuvre du processus (Process implementation) 11
I.2.2.6 - Suivi du processus (Process monitoring) 12
I.2.3 - Spécification des processus métier 12
I.2.3.1 - BPMN 12

I.2.3.2 - YAWL 13
I.2.3.3 - ADEPT 13
I.2.3.4 - AWGAG 13
I.2.4 - Exécution des processus métier 14
I.3 - Généralités sur le Développement web 15
I.3.1 - Historique du Développement web 15
I.3.2 - Définition d'application web et concepts du web 16
I.3.2.1 - Définition 16
I.3.2.2 - Concepts du web 16
I.3.3 - Frameworks d'application web 22
I.3.3.1 - Frameworks de développement web back-end 22
I.3.3.2 - Frameworks de développement web front-end 23
I.3.4 - Sécurité dans les applications web 23
I.4 - Conclusion 25

Chapitre II ▸ **État de l'art sur la spécification des processus métier pouvant s'exécuter au sein d'une application web** **26**
II.1 - Introduction 26
II.2 - Ingénierie Dirigée par les Modèles (IDM) 27
II.2.1 - Définition et principe 27
II.2.2 - Principaux concepts de l'IDM 29
II.2.2.1 - Les modèles 29
II.2.2.2 - Les transformations de modèles 30
II.3 - Business Process Management et industrie du Développement web . 30
II.3.1 - Approches traditionnelles d'ingénierie web pour la modélisation des applications web 32
II.3.1.1 - Méthodes orientées données 32
II.3.1.2 - Méthodes orientées hypertexte 32
II.3.1.3 - Méthodes orientées objet 32
II.3.1.4 - Méthodes orientées logiciel 33
II.3.2 - Synthèse et critique des approches 33
II.4 - Vision métier pour la modélisation des applications web 34
II.4.1 - Applications web basées sur les processus métier 34
II.4.2 - Rôle des processus métier dans l'analyse de besoins 35
II.4.3 - Vue d'ensemble sur les approches pour la gestion des processus métier dans les applications web 36

II.4.3.1 - An empirical study of rules for mapping BPMN models to graphical user interfaces 36
II.4.3.2 - Bizzy v2 36
II.4.3.3 - WebRatio BPM 37
II.4.3.4 - An Artifact-centric Approach to Generating webbased Business Process Driven User Interfaces . . . 37
II.4.3.5 - User-Centered Design and Business Process Modeling : Cross Road in Rapid Prototyping Tools . . . 37
II.4.3.6 - Bizzy v1 38
II.4.3.7 - BonitaSoft 38
II.4.3.8 - Node-RED 38
II.4.3.9 - Camunda BPM 38
II.4.3.10 - ASSEMBLE : un outil de développement des processus métier collaboratifs 39
II.4.4 - Synthèse et critique des approches 40
II.5 - Conclusion 41

Chapitre III ▶ **Vers une approche basée sur l'Ingenierie Dirigée par les Modèles, de gestion des workflows dans une application web** **42**
III.1 - Introduction 42
III.2 - Architecture du système 43
III.2.1 - Application de base 43
III.2.1.1 - Paquetage Spring-Boot Core / ASP .NET Core . . . 44
III.2.1.2 - Paquetage RBAC 44
III.2.1.3 - Paquetage WMS 45
III.2.1.4 - Sous-paquetage Processes du paquetage WMS . . . 45
III.2.1.5 - Paquetage Process Library 46
III.2.2 - Cycle de vie de développement classique 47
III.2.3 - Cycle de vie de développement des processus 48
III.3 - Extension de *LSAWfP* pour la modélisation des workflows du web : le langage *LSAWfP4WEB* 49
III.3.1 - Motivation du choix de *LSAWfP* pour la modélisation des workflows du web 50
III.3.2 - Introduction au langage *LSAWfP4WEB* 50
III.3.3 - Présentation du processus de réclamation 51
III.3.4 - Etapes préliminaires de modélisation du processus de réclamation avec *LSAWfP4WEB* 52

III.3.5 - Modèle de workflow du langage *LSAWfP4WEB* 52
III.3.6 - Etapes finales de modélisation du processus de réclamation avec *LSAWfP4WEB* 54
III.4 - Méta-modèle du langage *LSAWfP4WEB* 57
III.5 - Conclusion 57

Conclusion générale **58**
Bilan du travail 58
Quelques perspectives 59

Bibliographie **60**

LISTE DE SYMBOLES

$\mathbb{G}$	Modèle grammatical de Workflow ;
$\mathbb{W}_f$	Workflow Administratif ;
$\mathcal{L}_{P_k}$	Liste des acteurs ;
$\mathcal{L}_{\mathcal{A}_k}$	Liste des accréditations associées aux acteurs ;
$\mathcal{S}$	Ensemble des symboles de la grammaire ;
$\mathcal{P}$	Ensemble des productions de la grammaire ;
Ent	Ensemble des entités de la grammaire ;
$\mathcal{A}$	Ensemble des axiomes de la grammaire ;
P_r	Ensemble des permissions pour la lecture des données d'une tâche
P_x	Ensemble des permissions pour l'exécution d'une tâche

LISTE DES ACRONYMES

API	Application Programming Interface
BP	Business Process
BPM	Business Process Managment
BPMN	Business Process Model and Notation
CSS	Cascading Style Sheets
DSEL	Domain Specific Embedded Language
GUI	Graphical User Interface
HDM	Hypermedia Design Methodology
HFPM	Hypermedia Flexible Process Modeling
HTML	HyperText Markup Language
IDM	Ingenierie Dirigée par les Modèles
JSON	JavaScript Object Notation
LSAWfP	A Language for the Specification of Administrative Workflow Processes
LSAWfP4WEB	A Language for the Specification of Administrative Workflow Processes for WEB
OOHDM	Object Oriented Hypermedia Design Model
OOWS	Object-Oriented Web-Solutions
OO-H	Object Oriented Hypermedia
P2P	Peer to Peer
RBAC	Role-Based Access Control
REST	REpresentational State Transfer
RMM	Relationship Management Methodology
SOHDM	Scenario-based Object Oriented Hypermedia Design Methodology
UWE	UML based Web Engeneering Methodology
WebML	Web Modelling Language
WMS	Workflow Management System
WSDM	Web Site Design Method

LISTE DES TABLEAUX

I - Liste exhaustive des tâches du processus de réclamation et leurs exécutants respectifs 52
II - Liste exhaustive des permissions pour l'accès aux données et pour l'exécution de châque tâches 55
III - Liste des tâches et groupes respectifs 56

TABLE DES FIGURES

1 - Les ingrédients de base d'un processus (source [1]) 9
2 - Le cycle de vie du BPM (source [1]) 10
3 - Flux côté serveur (source [2]) 17
4 - Flux côté client (source [2]) 18
5 - API REST 21
6 - Cœur du modèle RBAC (source [3]) 24
7 - Une famille de modèles de contrôle d'accès basés sur les rôles. (source [4]) 25

8 - Vue d'ensemble sur les approches traditionnelles de modélisation des applications web (source [5]) 33
9 - Composants principaux de Camunda BPM 39
10 - Architecture globale du système ASSEMBLE (source [6]) 40

11 - Architecture du système proposé 43
12 - Application de base 44
13 - Paquetages de l'application de base 45
14 - Paquetages d'un processus dans le moteurs de gestion de workflows . 46
15 - Cycle de vie de développement classique 47
16 - Cycle de vie de développement des processus 48
17 - Présentation des processus déployés au sein de l'application de base . 49
18 - Artefacts représentatifs du workflow de réclamation 55
19 - Modèle grammatical de workflow du processus de réclamation en utilisant le langage *LSAWfP4WEB* 56
20 - Méta-modèle du langage *LSAWfP4WEB* proposé 57

RÉSUMÉ

Le travail réalisé dans ce mémoire s'inscrit dans le domaine de la gestion des processus métier ; plus précisément, il contribue aux travaux sur la modélisation des processus métier et leur gestion (configuration, exécution, analyse, etc.) au sein d'applications web. En fait, dans les approches traditionnelles de création d'applications web, les processus métier automatisés ne sont généralement pas au centre du cycle de vie. Pourtant ce sont ces processus qui décrivent au mieux les exigences métier des organisations. Plus précisément, ils décrivent les acteurs, les activités ainsi que les informations qui circulent au sein de celle-ci. La négligence des exigences métier conduit à des problèmes de grande envergure parmi lesquels : le non-respect des délais de livraison et l'explosion des budgets consacrés au développement. Il est de ce fait nécessaire d'accorder une place importante aux exigences métier dans le cycle de vie de mise en œuvre des applications web. La littérature s'accorde sur le fait que l'utilisation des techniques développées dans le domaine de la gestion des processus métier pourrait apporter une solution à ce problème. Nous partageons cette vision des choses dans ce travail dont l'idée est de proposer un framework ; le but dudit framework est de faciliter l'implémentation des processus métier dans les applications web. Le but est de fournir au développeur web, une boite à outil lui permettant de modéliser ses processus métier à l'aide d'un langage de workflow puis, de dériver à partir des modèles obtenus, un code métier qu'il peut faire évoluer comme il en a l'habitude. Concrètement donc, nous proposons une architecture d'applications web facilitant la gestion des processus métier au moyen des méthodes d'Ingénierie Dirigée par les Modèles (IDM) et du Business Process Management (BPM). Nous proposons aussi un modèle grammatical pour la spécification des processus afin de faciliter leur exécution dans les applications web basées sur l'architecture proposée.

Mots clés : Processus Métier, Application Web, Business Process Management, Modélisation, Modèle Grammatical, Ingénierie Dirigée par les Modèles.

ABSTRACT

The work done in this thesis is part of the field of business process management ; more precisely, it contributes to business process modeling and management (configuration, execution, analysis, etc.) within web applications. In fact, in traditional approaches to building web applications, automated business processes are generally not at the center of the life cycle. Yet it is these processes that best describe the business requirements of organisations. More precisely, they describe the actors, activities and informations that circulate within the organisation. The neglect of business requirements leads to major problems such as the failure to meet delivery deadlines and the explosion of budgets devoted to development. It is therefore necessary to give an important place to business requirements in the implementation of the life cycle of the web applications. The literature agrees that the use of techniques developed in the field of business process management could provide a solution to this problem. We share this view in this work whose idea is to propose a framework ; the purpose of this framework is to facilitate the implementation of business processes in web applications. This goal is to provide the web developer with a toolbox allowing him to model his business processes using a workflow language and then to derive from the obtained models a business code that he can develop as usual. Concretely, we propose a web application architecture that facilitates the management of business processes using Model Driven Engineering (MDE) and Business Process Management (BPM) methods. We also propose a grammatical model for the specification of processes in order to facilitate their execution in web applications based on the proposed architecture.

Keywords : Business Process, Web Application, Business Process Management, Modeling, Grammatical Model Model Driven Engineering

INTRODUCTION GÉNÉRALE

SOMMAIRE

Contexte du travail 1
Problématique et objectifs 2
Organisation du manuscrit 3

Contexte du travail

Une grande partie des activités que nous menons sur internet peut être décrite en termes de processus métier. Que ce soit lorsqu'on fait un achat sur un site de e-commerce, lorsqu'on dépose un dossier en ligne pour un recrutement, lorsqu'on soumet une réclamation dans une agence, lorsqu'on suit une procédure de préinscription dans une université ou encore lorsqu'on soumet un article dans une revue scientifique ; nous sommes emmenés à exécuter des processus. En effet, chacune de ces activités concourent à l'atteinte d'un objectif spécifique. Certaines applications web sont typiquement conçues pour l'offre des services en ligne et ne font qu'exposer des procédures et activités que des *clients* viendront initier et suivre. C'est le cas par exemple des sites web des administrations, des banques, des impôts ou encore des plateformes de e-learning.

Pour la mise œuvre des applications web, plusieurs techniques de modélisations ont été proposées. On peut identifier principalement deux courants de modélisation des applications web [49] : la modélisation majoritairement centrée sur les aspects informatiques ou technologiques de l'application considérée comme la réponse aux besoins métier ; et la modélisation majoritairement centrée sur les besoins métier eux-mêmes.

Les approches de modélisation majoritairement centrée sur les aspects informatiques ou technologiques (RMM [51], Hera [52], WebML [53], HDM [54], etc.) présentent un manque d'accessibilité [49]. En effet, la finalité de ces approches va

du prototypage des interfaces utilisateurs à la génération partielle ou complète de l'application. Ces approches sont destinées, principalement, aux *concepteurs informaticiens* et se préoccupent davantage de la solution technique de l'application que du métier des utilisateurs.

Cette tendance à ne pas placer les exigences métier au centre de la modélisation a entraîné une cascade de problèmes ; une étude faite par le Cutter Consortium [48] a montré que seulement 16% des applications répondent aux besoins métier réels, les délais de livraison des applications ne sont pas respectés dans 79% des cas, les budgets consacrés au développement des applications sont dépassés dans 63% des cas, la pauvreté des spécifications fonctionnelles touche 53% des cas, la qualité des livrables est mauvaise dans 52%. Pour pallier à ces problèmes, la littérature s'est accordée sur la nécessité de placer les exigences métier au centre de la modélisation.

Les approches de modélisation majoritairement centrées sur les besoins métier ont introduit la modélisation explicite des processus métier au sein des applications web. Ceci dans le but de résoudre les problèmes identifiés par le Cutter Consortium [48]. Cependant, cette modélisation reste à un niveau secondaire par rapport à la modélisation de l'application web. En effet, la plupart des solutions BPM de modélisation des processus métier (LSAWfP [7], BPMN [15], YAWL [18], etc.) souffrent d'un manque d'expressivité vis à vis de leur utilisation pour la mise en œuvre des applications web. Dans ce cas, le passage par un modèle intermédiaire [65, 73, 75, 76, 6] est nécessaire afin de compléter la modélisation de l'application. De plus, l'utilisation des approches de modélisation centrées sur les besoins métier sont principalement destinées aux *ingénieurs BPM* qui ont un ensemble de connaissances sur les techniques de modélisation et de mise en œuvre des processus métier.

Ce travail se situe au juste milieu entre les approches de modélisation centrée sur les aspects informatiques et les approches de modélisation centrée sur les besoins métier. En effet, notre approche est destinée aux *concepteurs informaticiens* et aux *ingénieurs BPM*. Notre vision consiste à fournir un framework d'application web permettant la gestion des processus métier mais aussi du développement classique de fonctionnalités.

Problématique et objectifs

Nous pouvons formuler la problématique de ce travail de la manière suivante : comment fournir un ensemble de techniques ainsi qu'une méthodologie permettant

à un concepteur de systèmes d'information, de spécifier des processus avec une approche BPM et ensuite de générer les fonctionnalités correspondantes dans une application web de telle sorte qu'il puisse faire évoluer celle-ci grâce aux méthodes classiques connues ?

Notre objectif principal est de proposer une approche pour la modélisation des processus du web en vue de leur exécution dans une architecture d'application web fermée à la modification et ouverte à l'extension. Plus précisément, nous voulons :

— proposer une architecture logicielle pour un système de création d'applications web incluant la gestion des processus métier ;
— étendre le langage de modélisation des processus *LSAWfP* [7] (A Language for the Specification of Administrative Workflow Processes) pour pouvoir modéliser les processus intervenants au sein d'une application web ;
— proposer un DSEL (Domain Specific Embedded Language) basé sur JSON (JavaScript Object Notation) pour la création, la sauvegarde et le partage des spécifications du langage proposé.

Organisation du manuscrit

La suite de ce document est organisée comme suit :

Chapitre 1 : Généralités sur le Business Process Management (BPM) et le Développement web. Dans ce chapitre nous présentons quelques concepts de base liés aux domaines du BPM et du Développement web ;

Chapitre 2 : État de l'art sur la spécification des processus métier pouvant s'exécuter au sein d'une application web. Dans ce chapitre nous présentons et établissons une classification d'une part des méthodes traditionnelles d'ingénierie web pour la modélisation des applications web ; d'autre part des méthodes d'ingénierie web pour la gestion des processus métier au sein des applications web ;

Chapitre 3 : Vers une approche basée sur l'Ingénierie Dirigée par les Modèles, de gestion des workflows dans une application web. Dans ce chapitre nous proposons dans un premier temps une architecture d'application web permettant de faire à la fois de la gestion des processus métier mais aussi du développement classique. Dans un second temps, nous proposons une extension du langage *LSAWfP* [7] (A Language for the Specification of Administrative Workflow Processes) nommée *LSAWfP4WEB* pour la spécification des processus métier qui vont s'exécuter

au sein d'une application web. Enfin nous proposons un DSEL JSON de ce langage pour la création, la sauvegarde et le partage des spécifications.

Conclusion Générale : nous résumons ce travail et présentons quelques perspectives prometteuses pour son développement futur.

I

CHAPITRE

GÉNÉRALITÉS SUR LE BUSINESS PROCESS MANAGEMENT ET LE DÉVELOPPEMENT WEB

SOMMAIRE

I.1 - Introduction 5
I.2 - Généralités sur le Business Process Management (BPM) 6
I.3 - Généralités sur le Développement web 15
I.4 - Conclusion 25

I.1. Introduction

Le Business Process Management (BPM) est une discipline utilisant un ensemble de connaissances dans les domaines des technologies de l'information et des sciences de gestion pour une application de celles-ci aux processus opérationnels [8]. Un processus opérationnel ou encore processus métier se définit comme un ensemble ordonné de tâches à exécuter soit par des systèmes ou des humains pour atteindre un objectif métier. Cette discipline s'intéresse fondamentalement à l'analyse, la vérification, la gestion (maintenance), la spécification et l'automatisation des processus métier. C'est donc sur cette base que de nombreux outils techniques ont été développés. Aujourd'hui, les concepts de ce domaine sont utilisés et on dénombre de nombreux systèmes BPM nous facilitant la vie au quotidien. L'industrie du développement web quant à elle connaît une croissance très rapide depuis ses débuts au milieu des années 90. On assite au développement de multiples technologies pour le développement à la fois côté client et côté serveur. Au fil du temps, plusieurs frameworks d'applications web ont été créés ainsi que plusieurs

méthodes d'ingénierie web ainsi que d'IDM notamment dans le but d'atténuer les problèmes courants rencontrés lors de la création des applications web. Dans ce chapitre, nous allons de prime à bord présenter la discipline du Business Process Management tout en insistant sur son historique, les concepts de base, les étapes de construction d'une solution BPM ainsi que la spécification et l'exécution des processus métier. Enfin nous ferons une petite étude de la discipline du développement web partant de son historique, en passant par ces concepts et nous terminerons par présenter une des politiques de gestion du contrôle d'accès sur le web.

I.2. Généralités sur le Business Process Management (BPM)

Marlon Dumas et al. [1] définissent le Business Process Management (BPM) comme étant l'art et la science qui s'intéresse à la supervision de la façon dont le travail est effectué au sein d'une organisation dans le but de s'assurer de la cohérence des résultats et de profiter des opportunités d'amélioration. Comme exemples typiques d'objectifs d'amélioration figurent la réduction des coûts, la réduction des délais d'exécution et la réduction des taux d'erreur, mais aussi l'obtention d'un avantage concurrentiel par l'innovation. Le BPM permet la gestion des chaînes entières d'événements, d'activités et de décisions qui, au final, apportent une valeur ajoutée à l'organisation et à ses clients [8]. Ces chaînes d'événements, d'activités et de décisions sont appelées processus.

Les processus sont au cœur des activités des organisations. Ceux-ci décrivent au mieux les flux d'informations qui circulent, les acteurs mis en jeu ainsi que les tâches respectives que ceux-ci exécutent pour l'atteinte d'un objectif spécifique de l'organisation. La discipline du BPM met l'accent sur cet aspect de gestion des processus tout en fournissant un ensemble d'outils et de techniques de modélisation, de spécification et d'exécution de ceux-ci.

Dans cette section, nous présentons les concepts essentiels du BPM. Nous commençons par une description des processus typiques que l'on trouve dans les organisations contemporaines. Ensuite, nous discutons des ingrédients de base d'un processus métier et fournissons une définition du processus métier et du BPM. Afin de placer le BPM dans une perspective plus large, nous fournissons un aperçu historique de la discipline. Par après, nous examinons comment se déroule généralement une initiative de BPM dans une organisation. Cette discussion nous conduit à la définition d'un cycle de vie du BPM. Par la suite nous présentons en quoi consiste la spécification des processus métier ainsi que les langages de spécification des processus existants et nous terminons avec l'exécution de ces processus.

I.2.1. Définitions et Historique

I.2.1.1. Définitions

Les processus font partie intégrante des organisations, qu'il s'agisse d'une entreprise, d'une organisation à but non lucratif ou d'une agence gouvernementale, ceux-ci sont emmenés à gérer un certain nombre de processus. Les exemples typiques de processus que l'on peut trouver dans la plupart des organisations sont les suivant :

— *la réclamation* (issue-to-resolution). Ce type de processus commence lorsqu'un client soulève un problème ou une question, comme une plainte liée à un défaut dans un produit ou un problème rencontré lors de la consommation d'un service. Le processus se poursuit jusqu'à ce que le client, le fournisseur, ou de préférence les deux, conviennent que le problème a été résolu. Une variante de ce processus peut être trouvée dans les compagnies d'assurance qui doivent traiter les réclamations d'assurance. Cette variante est appelée *claim-to-resolution*.

— *de la demande à l'approbation* (application-to-approval). Ce type de processus commence lorsqu'une personne demande un avantage ou un privilège et se termine lorsque l'avantage ou le privilège en question est accordé ou refusé. Ce type de processus est courant dans les agences gouvernementales, par exemple lorsque les citoyens demandent un permis de construire ou lorsque les entrepreneurs demandent une licence commerciale (par exemple, pour ouvrir un restaurant). Un autre processus qui entre dans cette catégorie est le processus d'admission dans une université, qui commence lorsqu'un étudiant demande à être admis dans un programme diplômant. Un autre exemple encore est le processus d'approbation des demandes de vacances ou de congés spéciaux dans une entreprise.

— *de la commande à l'encaissement* (order-to-cash). Il s'agit d'un type de processus réalisé par un fournisseur, qui commence lorsqu'un client soumet une commande pour l'achat d'un produit ou d'un service et se termine lorsque le produit ou le service en question a été livré au client et que ce dernier a effectué le paiement correspondant. Un processus *order-to-cash* englobe les activités liées à la vérification du bon de commande, à l'expédition (pour les produits physiques), à la livraison, à la facturation, à la réception du paiement et à l'accusé de réception.

— *du devis à la commande* (quote-to-order). Ce type de processus précède généralement le processus *order-to-cash*. Il commence au moment où un

fournisseur reçoit une demande de devis d'un client et se termine lorsque le client en question passe une commande sur la base du devis reçu. Le processus *order-to-cash* prend le relais à partir de ce moment-là. La combinaison d'un devis à la commande et du processus correspondant de la commande à l'encaissement est appelée devis à l'encaissement ou encore processus *quote-to-cash*.

Un processus métier englobe un certain nombre d'événements et d'activités. Les événements correspondent à des choses qui se produisent de manière atomique, ce qui signifie qu'ils n'ont pas de durée. La soumission d'une réclamation est un événement. Cet événement peut déclencher l'exécution d'une série d'activités. Par exemple, lorsqu'une réclamation est déposée, un employé l'inspecte dans le but de fournir une réponse appropriée au client. Cette inspection est une activité, dans le sens où elle prend du temps.

Lorsqu'une activité est plutôt simple et peut être considérée comme une seule unité de travail, celle-ci est considérée comme étant une tâche. Par exemple, si l'inspection de la réclamation du client est simple (il suffit de vérifier que la réclamation concerne un service offert par l'entreprise) nous pouvons dire qu'il s'agit d'une tâche. Si, par contre, l'inspection de la réclamation du client implique plusieurs vérifications, comme par exemple vérifier que le client est enregistré dans le système, vérifier que le client a effectivement reçu le service pour lequel il n'est pas satisfait, nous l'appellerons une activité plutôt qu'une tâche. En d'autres termes, le terme tâche désigne une unité de travail à grain fin exécutée par un seul participant au processus, tandis que le terme activité est utilisé pour désigner des unités de travail à la fois à grain fin et à grain grossier.

En plus des événements et des activités, un processus typique comprend des points de décision, c'est-à-dire des moments où une décision prise affecte la façon dont le processus est exécuté. Par exemple, à la suite de l'inspection de la réclamation, l'employé de la boite peut décider que la réclamation est valable ou pas en ce sens qu'elle concerne effectivement un service offert par l'organisation. Cette décision affecte ce qui se passe plus tard dans le processus.

Un processus implique également :

— *des acteurs*, incluant les acteurs humains, les organisations ou les systèmes logiciels agissant au nom d'acteurs humains ou d'organisations.

— *les objets physiques*, tels que les équipements, les matériaux, les produits, les documents papier.

— *les objets informationnels*, tels que les documents électroniques et les enregistrements électroniques.

Enfin, l'exécution d'un processus conduit à un ou plusieurs résultats. Par exemple, le processus de réclamation conduit à un résultat positif, cas ou le client est satisfait de la réponse de l'employé face à sa préocupation ; ou à un résultat négatif, cas ou le client n'est pas satisfait de la réponse fournie par l'employé.

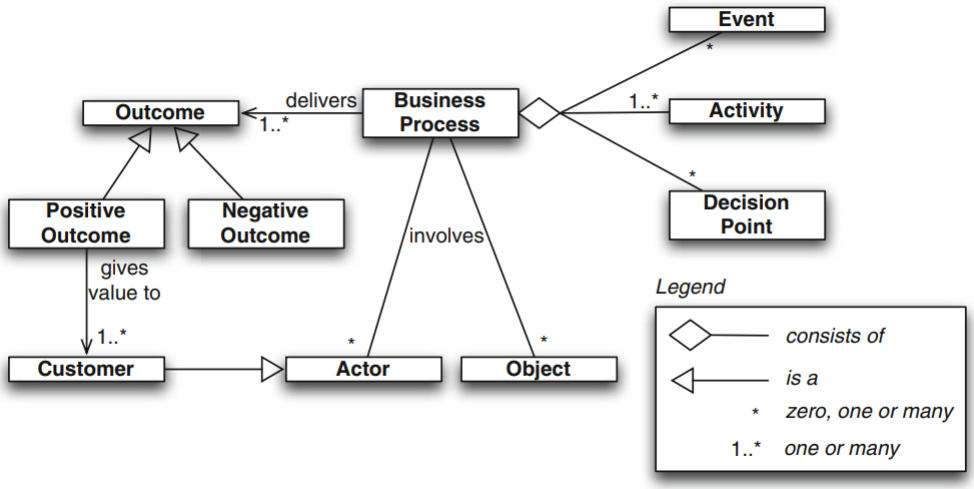


FIGURE 1 – *Les ingrédients de base d'un processus (source [1])*

A la lumière de ce qui précède, nous définissons un processus métier comme un ensemble d'événements, d'activités et de points de décision interdépendants qui impliquent un certain nombre d'acteurs et d'objets, et qui conduisent collectivement à un résultat ayant de la valeur pour au moins un client. La figure 1 illustre les ingrédients de cette définition et leurs relations.

I.2.1.2. Historique du BPM

Historiquement, le BPM apparait dans les années 1980 sous une forme plus simple appelée workflow Management (WfM). Auparavant, les systèmes d'information étaient construits à partir de zéro ; tous les composants de ces systèmes devaient être programmés [11]. Les fournisseurs de logiciels se sont vite rendu compte que de nombreux systèmes d'information avaient des exigences similaires en matière de gestion des données. Les fonctionnalités génériques ont donc été externalisées vers un système de gestion des données. Par la suite, la fonctionnalité générique liée à l'interaction avec l'utilisateur a été confiée à des générateurs d'interfaces utilisateur. La tendance à l'externalisation des fonctionnalités récurrentes vers des outils génériques s'est poursuivie dans différents domaines. C'est dans ce contexte que le WfM a été introduit.

Au fil du temps, le WfM a évolué vers le BPM. Le BPM tient ses fondements sur l'analyse, la vérification et la maintenance, la spécification et l'automatisation des processus [12]. Avec cette évolution, de nombreux outils et techniques ont été développés et ont permis au domaine du BPM de grandir considérablement. Aujourd'hui, sa pertinence est reconnue par les praticiens (utilisateurs, managers, analystes, consultants et développeurs de logiciels) et les universitaires. Les concepts de ce domaine sont utilisés et on dénombre de nombreux systèmes BPM. Notamment des outils capables de générer automatiquement des interfaces utilisateur à partir des spécifications de processus ou encore des générateurs d'applications web.

I.2.2. Cycle de vie du BPM

Selon Marlon Dumas [1], le BPM peut être considéré comme un cycle continu comprenant les phases suivantes (figure 2) :

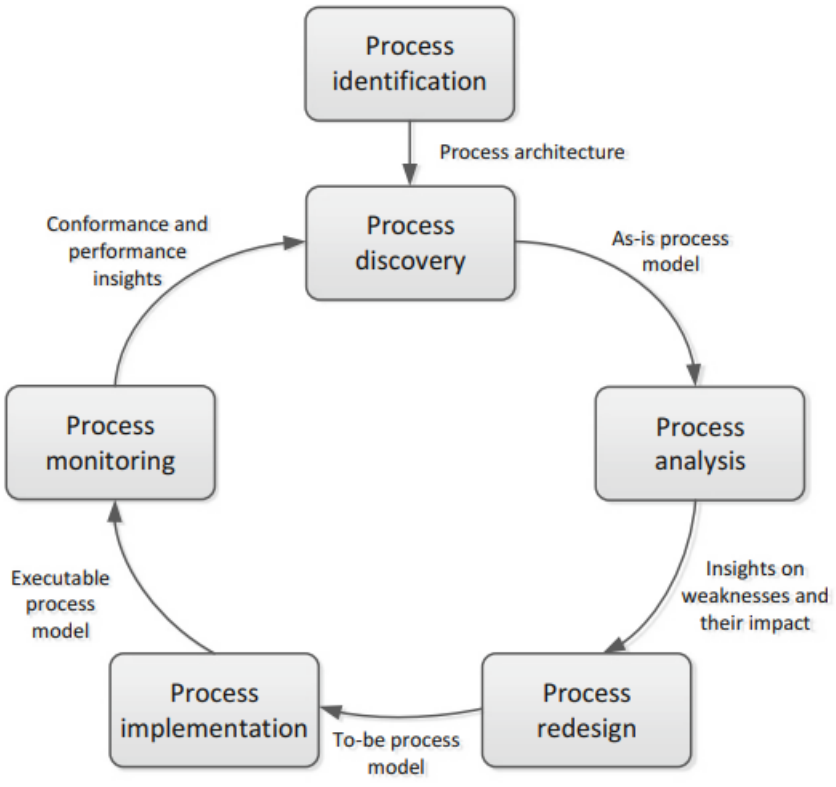


FIGURE 2 – *Le cycle de vie du BPM (source [1])*

I.2.2.1. Identification du processus (Process Identification)

Dans cette phase, un problème d'entreprise est posé. Les processus pertinents pour le problème à traiter sont identifiés, délimités et reliés entre eux. Le résultat de

l'identification des processus est une architecture de processus nouvelle ou mise à jour, qui fournit une image globale des processus dans une organisation et de leurs relations. Cette architecture est ensuite utilisée pour sélectionner le processus ou l'ensemble de processus à gérer au cours des phases restantes du cycle de vie. Généralement, l'identification des processus se fait en parallèle avec l'identification des mesures de performance.

I.2.2.2. Découverte des processus (Process discovery)

Également appelé modélisation des processus. Ici, l'état actuel de chacun des processus pertinents est documenté, généralement sous la forme d'un ou plusieurs modèles de processus.

I.2.2.3. Analyse du processus (Process analysis)

Dans cette phase, les problèmes associés au processus sont identifiés, documentés et, si possible, quantifiés à l'aide de mesures de performance. Le résultat de cette phase est un ensemble structuré de problèmes. Ces problèmes sont classés par ordre de priorité en fonction de leur impact potentiel et de l'effort estimé nécessaire pour les résoudre.

I.2.2.4. Refonte du processus (Process redesign)

Également appelée amélioration des processus. Ici l'objectif de cette phase est d'identifier les changements à apporter au processus qui contribueraient à résoudre les problèmes identifiés lors de la phase précédente et permettraient à l'organisation d'atteindre ses objectifs de performance. À cette fin, plusieurs options de changement sont analysées et comparées en fonction des mesures de performance choisies. Ainsi, la refonte et l'analyse des processus vont de pair : Au fur et à mesure que de nouvelles options de changement sont proposées, elles sont analysées à l'aide de techniques d'analyse des processus. Finalement, les options de changement les plus prometteuses sont retenues et combinées dans un processus remanié. Le résultat de cette phase est généralement un modèle de processus amélioré.

I.2.2.5. Mise en œuvre du processus (Process implementation)

Dans cette phase, les changements nécessaires pour passer du processus actuel au processus futur sont préparés et réalisés. La mise en œuvre des processus couvre deux aspects : la gestion du changement organisationnel et l'automatisation. La gestion du changement organisationnel fait référence à l'ensemble des activités nécessaires pour modifier la façon de travailler de tous les participants impliqués dans le processus. L'automatisation des processus fait référence au développement

et au déploiement de systèmes informatiques (ou de versions améliorées de systèmes informatiques existants) qui prennent en charge le processus futur.

I.2.2.6. Suivi du processus (Process monitoring)

Une fois que le processus remanié est en cours d'exécution, les données pertinentes sont collectées et analysées pour déterminer la performance du processus par rapport à ses mesures et objectifs de performance. Les goulots d'étranglement, les erreurs récurrentes ou les déviations par rapport au comportement prévu sont identifiés et des actions correctives sont entreprises. De nouveaux problèmes peuvent alors apparaître, dans le même processus ou dans d'autres, ce qui nécessite de répéter le cycle en permanence.

I.2.3. Spécification des processus métier

Après la phase d'identification des processus de l'organisation, viens la phase de spécification de ceux-ci. Pendant cette phase il est question de représenter de manière formelle et à l'aide de schémas les différents processus ainsi que les éléments les constituant. En effet la spécification d'un processus correspond à la construction d'un modèle de workflow [7]. Selon [13], un modèle de workflow est constitué par trois modèles conceptuels principaux : le modèle organisationnel, le modèle informationnel et le cycle de vie. Le modèle organisationnel permet de représenter et de classer les ressources responsables de l'exécution des tâches du processus étudié. Généralement, celles-ci sont classées en rôles et/ou groupes auxquels sont attribués des tâches. Le modèle informationnel décrit les données qui transitent (données consommées et données produites) lors de l'exécution des processus. Enfin, le cycle de vie décrit la structure des tâches tout en fournissant les informations telles que l'ordre d'exécution des tâches et la coordination entre les différents acteurs impliqués dans leur exécution. Généralement, le cycle de vie est exprimé à l'aide d'un langage et permet l'expression des flux de contrôle de base entre les tâches tel que les flux de contrôle séquentiel, parallèle, alternatif et itératif. Idéalement, pour obtenir une spécification complète, un langage de workflow devrait pouvoir permettre aux concepteurs de modèles de workflow d'exprimer ces trois modèles conceptuels [7]. Il existe plusieurs langages de modélisation des processus.

I.2.3.1. BPMN

La notation standard de modélisation des processus métier (BPMN) a été développée par la Business Process Management Initiative (BPMI). La spécification BPMN 1.0 a été rendue publique en mai 2004 et a évoluée pour passer à la spécifi-

cation BPMN 2.0 en 2011 [14]. BPMN fournit une notation facilement compréhensible par tous les utilisateurs professionnels, depuis les analystes métier qui créent les premières ébauches des processus, jusqu'aux développeurs techniques responsables de la mise en œuvre de la technologie qui exécutera ces processus, et, enfin, jusqu'aux professionnels qui géreront et surveilleront ces processus [15]. Dans le BPMN, les *activités* sont représentées par des rectangles arrondis. Les *nœuds de contrôle* (appelés passerelles) sont représentés à l'aide de losanges. Les activités et les nœuds de contrôle sont reliés par des *arcs* (appelés flux de séquence) qui déterminent l'ordre dans lequel le processus est exécuté [17].

I.2.3.2. YAWL

Le langage *YAWL (Yet Another workflow Language)* a été proposé par Van der Aalst en 2005 [18] mais celui-ci ne connait pas autant de succès que le BPMN. En effet YAWL est basé sur les réseaux de Petri et étendu avec des fonctionnalités supplémentaires pour modéliser facilement les workflows complexes. Dans YAWL, les spécifications de workflow sont un ensemble de réseaux de workflow étendus (EWF-nets) qui forment une hiérarchie, en d'autres termes une structure arborescente. Les tâches peuvent être atomiques ou composites. Chaque tâche composée est associée à un seul réseau EWF au niveau inférieur de la hiérarchie. Les tâches atomiques constituent les feuilles de l'arborescence. La racine de l'arbre est un EWF-net appelé workflow de niveau supérieur.

I.2.3.3. ADEPT

ADEPT (Application Development based on Encapsulated Pre-modeled Process Templates) est un grand projet visant à l'amélioration de la gestion des processus. La technologie ADEPT offre des caractéristiques et des propriétés avancées au sein d'un seul système et qui sont nécessaires pour le support d'un large éventail de processus : facilité d'utilisation pour les utilisateurs finaux et les développeurs de systèmes, grande flexibilité, mise en œuvre rapide des modifications sur les processus grâce à l'évolution du schéma de processus, et garanties de correction permettant une exécution robuste des processus mis en œuvre [19].

I.2.3.4. AWGAG

Proposé par Badouel et al. [20], *AWGAG* est un modèle de processus basé les grammaires attribuées gardées (GAG). Les GAGs sont une collection de règles sémantiques qui décrivent comment produire des données à partir des informations fournies par un environnement. Les concepts de base de ce langage sont : *case*

ou cas qui désigne une instance du processus et est associé à un document qu'on appelle *artefact*. Nous avons aussi les *participants* qui représentent les acteurs, les intervenants du processus et enfin, les différentes règles, qui régissent l'exécution du processus, définies dans les *productions* de la grammaire attribuée associée par les règles sémantiques.

I.2.4. Exécution des processus métier

Plusieurs approches pour l'exécution des workflows ont été proposées. Principalement, on est passé des systèmes centralisés mis en œuvre selon un modèle architectural de référence [21], à des systèmes entièrement distribués pour l'exécution décentralisée des workflows [22, 23, 24]. De nouveaux paradigmes de spécification et de gestion des processus métier ont également été développés. Parmi ceux-ci, on trouve le modèle centré sur le document (document-centric [25, 26, 24]), celui centré sur la base de donnée (database-based [27, 28]), celui centré sur les données (data-centric [20, 29]), celui centré sur l'artefact (artifact-centric [30, 31, 32]), etc. Au cours des deux dernières décennies, le paradigme artifact-centric a été le sujet de nombreuses études et il a été très fructueux car il a permis le développement de langages de workflow beaucoup plus flexibles et mettant un accent sur les données des processus, contrairement aux langages existants comme le BPMN ou YAWL qui ne s'intéressaient qu'à l'ordonnancement des tâches du processus et à leur affectation aux acteurs.

Les modèles centrés sur les artefacts peuvent être exécutés par des WfMS (workflow Management Systems) centrés sur les artefacts. Deux tendances majeures dans l'approche de modélisation centrée sur l'artefact ont été développées : l'orchestration et la chorégraphie [30, 33].

1. L'*orchestration* suggère la création de systèmes centralisés (généralement appelés artefact hubs), coordonnés par un orchestrateur dont le rôle est de faciliter l'interaction entre les acteurs tout en s'assurant que les objectifs métier sont atteints.
2. Les approches orientées *chorégraphie* se débarrassent de l'orchestrateur et modélisent les acteurs comme des agents autonomes qui se coordonnent avec les artefacts et communiquent de manière P2P (Peer to Peer), afin d'atteindre les objectifs métier. Dans ces approches, chaque agent se concentre sur la réalisation d'un objectif local et la réalisation de l'objectif global est le résultat de l'agrégation des résultats des différents objectifs locaux.

À présent que nous avons présenté les bases de la discipline BPM, nous allons dans la prochaine section définir les concepts de base du développement web.

I.3. Généralités sur le Développement web

De manière générale, le développement web désigne l'ensemble des tâches associées à la conception, à l'implémentation, au déploiement et à la maintenance des applications web. Ces applications, une fois déployées, constituent des outils de stockage, de traitement et de partage de l'information auprès des utilisateurs de par le monde. C'est d'ailleurs l'une des premières motivations ayant conduit à la création du web. C'est dans ce sens que Tim Berners-Lee [34] a décrit l'idée selon laquelle *le rêve derrière le web est celui d'un espace d'information commun dans lequel nous communiquons en partageant des informations*. Dans cette section, nous rappelons les concepts de base du développement web ainsi qu'un ensemble de concepts un peu plus poussés de cette discipline.

I.3.1. Historique du Développement web

Les origines du développement web remontent à la création du web par le chercheur britannique Tim Berners-Lee [34] en 1989 lorsqu'il travaillait au CERN (Conseil Européen pour la Recherche Nucléaire). À l'origine, le projet a été conçu et développé pour que des scientifiques travaillant au sein des universités et instituts du monde entier puissent se partager des informations instantanément. L'idée de base du web est de combiner les technologies des ordinateurs personnels, des réseaux informatiques et de l'hypertexte pour créer un système d'information mondial, puissant et facile à utiliser.

C'est sur cette base que l'industrie du développement web a connu une croissance très rapide. Plusieurs technologies pour le codage à la fois côté client et côté serveur ont été développées. Nous sommes passés du web 1.0 ne proposant que des contenus statiques [35] au web 2.0 intégrant le dynamisme côté client et côté serveur, puis au web 3.0 encore appelé web sémantique et proposant non seulement des contenus dynamiques mais plus important encore l'étiquetage des contenus [35] en vue de leurs compréhension et interprétation par les moteurs d'indexation et autres technologies innovantes d'internet.

L'évolution du web s'est accompagnée de l'évolution des technologies de développement et des méthodologies d'Ingénierie web. Ainsi, plusieurs frameworks d'applications web ont été créés notamment dans le but d'atténuer les problèmes courants rencontrés lors de la création des applications web. En effet, les développeurs d'applications web sont confrontés aux mêmes problèmes que les développeurs d'applications de bureau. Ces problèmes sont notamment liés au temps et

au coût d'implémentation, aux méthodologies de conception, aux performances, à l'ergonomie, à la séparation de concepts, à la maintenance, à l'efficacité ...

I.3.2. Définition d'application web et concepts du web

I.3.2.1. Définition

HANS-W et al. [36] définissent une application web comme étant toute application logicielle qui dépend du web pour son exécution correcte. Manifestement, les logiciels explicitement conçus pour être distribués sur le web relèvent de cette définition, comme les sites web ou les journaux en ligne. Ces types de logiciels possèdent une solide notion de contenu. Nous incluont également les logiciels qui utilisent l'infrastructure web pour pouvoir s'exécuter. Par exemple, de nombreux systèmes d'information qui ont été conçus et construits avant le web sont maintenant disponibles sous forme d'applications web grâce à l'utilisation des navigateurs.

Au-delà des systèmes d'information hérités, il existe également des logiciels qui sont moins typiquement destinés à être diffusés sur le web mais qui en dépendent toujours en tant que plate-forme client-serveur pour leur exécution ; nous prenons le cas des applications web mobiles telles que Facebook, WhatApp, Instagram adaptées aux technologies mobiles comme les téléphones portables et qui se comportent comme des clients web.

I.3.2.2. Concepts du web

Il existe plusieurs architectures sur lesquelles sont basées les applications web. Il est donc important de choisir le bon type de modèle architectural pour optimiser l'utilisation et créer une solution réutilisable afin d'anticiper les problèmes courants survenant lors de la mise en œuvre ou dans un contexte donné. Ainsi, certaines architectures prennent en compte l'indépendance entre l'application cliente et l'application serveur considérant les deux comme des entités autonomes qui peuvent communiquer. D'autres ne prennent pas en compte cette séparation considérant que l'application web est un bloc unique à l'intérieur duquel on retrouve des instructions de traitement côté serveur au sein des vues comme c'est parfois observable avec les JSP (Java Servlet Pages) ou encore lorsqu'on développe des pages web avec du PHP. Dans cette section nous allons présenter quelques architectures sur lesquels sont basées les applications web ainsi que le design pattern MVC qui est très utile pour l'architecture des systèmes logiciels interactifs.

Le rendu coté serveur

En pratique, le rendu côté serveur (SSR pour Server Side Rendering) est une méthode de développement d'applications basées sur le web qui traite les demandes de l'utilisateur au niveau du serveur puis retourne les ressources sous un format interprétable par le navigateur de l'utilisateur (généralement du code HTML, CSS, Javascript ...). Cette méthode correspond généralement aux applications multi-pages.



FIGURE 3 – *Flux côté serveur (source [2])*

Le cycle requêtes-réponses entre le client et le serveur est le suivant :

— le client fait une demande initiale pour ouvrir un site web ou une application particulière.
— le serveur traite la demande puis répond avec des pages HTML, des images, du CSS, des fichiers de script et d'autres ressources externes.
— lorsque le client interagit avec une page, par exemple en cliquant sur un bouton ou en naviguant vers une autre page, il envoie une nouvelle demande au serveur.
— le serveur répond à nouveau avec une page HTML entière et non une partie particulière de la page.
— du côté du client, cette page est rechargée, ce qui nécessite plus de temps pour ces interactions client-serveur. Cela entraîne une augmentation de la bande passante et une expérience utilisateur moyenne.

Il existe de nombreux langages de programmation qui implémentent le concept de SSR parmi lesquels PHP ou Java.

Le rendu côté client

Parallèlement, le rendu côté client (CSR pour Client Side Rendering) est une méthode de développement d'applications basées sur le web pour laquelle les pages de l'application sont gérées dans le navigateur de l'utilisateur. Lors de la première requête, un bundle contenant l'ensemble des pages de l'application est chargé dans le navigateur de l'utilisateur. Le navigateur va extraire les pages du bundle puis les

interpréter et les afficher dynamiquement en fonction de l'interaction de l'utilisateur.



FIGURE 4 – *Flux côté client (source [2])*

Le cycle requêtes-réponses entre le client et le serveur est le suivant :

— le client fait une demande initiale pour ouvrir un site web ou une application particulière.
— le serveur traite la demande puis répond avec des pages HTML, des images, du CSS, des fichiers de script et d'autres ressources externes.
— lorsque le client interagit avec une page, par exemple en cliquant sur un bouton ou en naviguant vers une autre page, il envoie une nouvelle demande au serveur.
— le serveur répond maintenant uniquement avec le résultat de l'action de l'utilisateur et non plus avec la page HTML entière. C'est-à-dire que le serveur répond avec un JSON (JavaScript Object Notation) ou du XML ou tout autre format de transport de données qui contient seulement le résultat de l'action de l'utilisateur.
— côté client, la page n'est pas rechargée, mais une partie particulière de la page est remplacée/mise à jour avec cette réponse. Par conséquent, le cycle requêtes-réponses client-serveur nécessite très peu de bande passante et de temps. Cela conduit à une interface utilisateur très fluide.

Le CSR est la méthode employée dans les applications web monopage ou SPA (Single Page Application).

Les applications web monopages ou SPA

Une SPA (Single Page Application) est une application web qui charge entièrement toutes les ressources lors de la première requête, puis les composants de la page sont remplacés par d'autres composants en fonction de l'interaction avec l'utilisateur [37]. Si nous faisons une comparaison entre une SPA et une application web multipages, nous pouvons constater qu'il existe une analogie entre les "*états*" d'une SPA et les "*pages web*". La navigation des "*pages web*" dans une application web multipages est analogue à la navigation des "*états*" dans la SPA. Une monopage (SPA) est composée de composants individuels qui peuvent être

remplacés/mises à jour indépendamment, sans rafraîchir/recharger la page entière à chaque action de l'utilisateur.

— Composants individuels : la page entière est divisée en composants plus petits qui interagissent les uns avec les autres.
— Remplacé/mis à jour : un composant ou une région ou une partie de la page remplacé(e) ou mis(e) à jour en fonction des demandes des utilisateurs.
— Rafraîchissement/rechargement : la page entière n'est jamais rechargée/rafraîchie, mais le nouveau contenu est chargé dans une partie ou une section en fonction des nouvelles données.
— Actions de l'utilisateur : une SPA est chargée de traiter très rapidement toutes les actions de l'utilisateur, telles que les clics sur les boutons, les saisies au clavier, etc., ce qui donne lieu à une interface utilisateur très fluide.

Les SPA permettent de traiter les données de manière plus souple et plus élégante. Le rafraîchissement d'une partie ou d'une section particulière d'une page sans rafraîchir la page entière est l'objectif principal des SPA conduisant à une meilleure expérience utilisateur.

Lorsque nous construisons un SPA avec JavaScript par exemple, le front-end devient très complexe. Dans les grands projets, de nombreux développeurs travaillent en collaboration pour construire le front-end. Si le code est très difficile à comprendre parce qu'il n'y a pas de séparation entre la mise en page et la logique interne, la maintenance de ce code devient très difficile. Ce problème de maintenance du code des grands projets est résolu si l'application monopage utilise le design pattern MVC (Modèle Vue Contrôleur). Un code facile à lire et à comprendre devient la clé d'une efficacité élevée ainsi que d'une interface utilisateur fluide.

Le design pattern MVC

Le célèbre patron de conception Modèle/Vue/Contrôleur (ou MVC) est un moyen utile d'architecturer des systèmes logiciels interactifs [38]. L'idée clé est de séparer les interfaces utilisateur des données sous-jacentes qui y sont représentées. Dans le modèle MVC, la vue affiche des informations à l'utilisateur et le contrôleur traite l'interaction de l'utilisateur en se servant des modèles qui décrivent les données manipulées.

Les applications web et le design pattern MVC

Le problème de l'utilisation du modèle MVC pour développer des applications web vient du fait que les applications web sont intrinsèquement divisées entre le client et le serveur. La vue, bien sûr, est affichée sur le client, mais le modèle et le contrôleur peuvent (théoriquement) être répartis de n'importe quelle manière entre le client et le serveur en fonction de la méthode employée (Rendu côté client ou rendu côté serveur). Le développeur est obligé de partitionner l'application web "en amont" ; certainement au stade de la mise en œuvre, et souvent dès la phase de conception. En revanche, MVC est indépendant de la partition : c'est-à-dire que le modèle, la vue et le contrôleur résident et s'exécutent dans un espace d'adresse unique dans lequel les problèmes de partitionnement ne se posent pas. L'indépendance vis-à-vis de la partition est l'une des caractéristiques de MVC, car les questions liées à l'emplacement ne devraient pas déterminer les décisions d'architecture ou de conception. Malheureusement, le cloisonnement implique que les applications web sont dépendantes de l'emplacement, et cette caractéristique signifie qu'il est beaucoup plus difficile mais pas impossible d'appliquer le modèle MVC dans le contexte des applications web.

En bref, les applications web peuvent certainement utiliser le modèle de conception MVC lorsque :

— le partitionnement correct est connu et
— l'infrastructure technologique disponible est compatible avec ce cloisonnement [38].

La norme REST

REST (REpresentational State Transfer) est un style architectural que Roy T. Fielding a défini pour la première fois dans sa thèse de doctorat. Pour comprendre REST, il est nécessaire de comprendre la définition de ressource, de représentation et d'état.

Une ressource peut être n'importe quoi. Une ressource peut être un objet physique ou un concept abstrait. Tant qu'une chose est suffisamment importante pour être référencée comme une chose en soi, elle peut être exposée comme une ressource [40]. En général, une ressource est quelque chose qui peut être stocké sur un ordinateur et représenté sous forme de flux de bits.

Une représentation est toute information utile sur l'état d'une ressource. Une ressource peut avoir plusieurs représentations différentes.

Dans REST, il existe deux types d'état. L'un est l'état de la ressource, qui est une information sur une ressource, et l'autre est l'état de l'application, qui est une information sur le chemin que le client a emprunté dans l'application. L'état de la ressource reste sur le serveur et l'état de l'application ne vit que sur le client. REST fournit un ensemble de contraintes architecturales qui, lorsqu'elles sont appliquées dans leur ensemble, mettent l'accent sur l'évolutivité des interactions entre les composants, la généralité des interfaces, le déploiement indépendant des composants et les composants intermédiaires pour réduire la latence des interactions, renforcer la sécurité et encapsuler les systèmes existants. Les caractéristiques principales de REST sont les suivantes :

— tout est ressource ;
— les ressources sont identifiées par des URI (Universal Ressource Identifier) ;
— l'interface est uniforme ;
— la manipulation des ressources se fait par des représentations ;
— les messages sont autodescriptifs ;
— les interactions sont sans état.

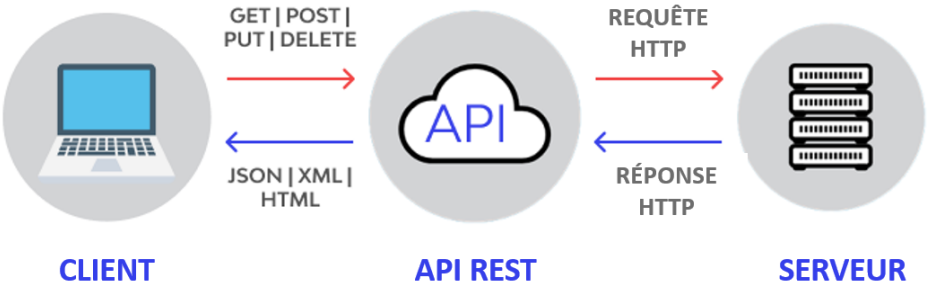


FIGURE 5 – *API REST*

Une ressource est un concept nominal, de sorte que la modélisation basée sur REST est nominalement centralisée. La Figure 5 donne l'illustration de l'architecture des services web RESTful (Cas d'une API). Le client interagit avec le serveur par l'intermédiaire d'une interface uniforme et, pendant l'interaction sans état, le serveur et le client échangent des représentations de ressources. La mise en œuvre de REST est le protocole HTTP (HyperText Transfert Protocol). REST est formé sur la base d'une véritable compréhension du protocole HTTP. Dans REST, HTTP est un protocole de transfert d'états plutôt qu'un protocole de transport de données. HTTP permet non seulement de localiser une ressource de manière unique, mais aussi de nous indiquer comment exploiter cette ressource. REST exige qu'une re-

quête et une réponse soient effectuées par le biais de quatre opérations HTTP : GET, POST, PUT et DELETE. GET récupère l'état actuel d'une ressource dans une représentation quelconque, POST crée une nouvelle ressource, PUT transfère un nouvel état sur une ressource et DELETE supprime une ressource existante. Chaque méthode a une sémantique attendue pour décider quelles méthodes sont appropriées pour chaque ressource. La contrainte de l'interface uniforme agit comme une sorte de règle régissant la communication client-serveur pour l'accès aux ressources. Elle permet de découpler le client et le serveur et fait que le client et le serveur peuvent évoluer indépendamment. Tant que l'interface reste inchangée, le client et le serveur peuvent interagir normalement. Ces avantages de REST font en sorte qu'il est possible d'implémenter le design pattern MVC dans une application web constitué d'un client léger SPA et un serveur REST par exemple.

I.3.3. Frameworks d'application web

La définition du dictionnaire du terme "framework" comme "structure de base sous-jacente à un système" n'est pas très éloignée de la compréhension de ce qu'est un framework en programmation. Dans l'industrie du développement de logiciels, un framework est un outil qui fournit des composants prêts à l'emploi ou des solutions personnalisées afin d'accélérer le développement. Celui-ci constitue une base pour le développement de logiciels. Les ingénieurs logiciels et les développeurs utilisent les frameworks comme modèle afin de créer des applications de plusieurs types.

Compte tenu de la grande variété de langages de programmation existants, il existe naturellement un grand nombre de frameworks disponibles. Chacun de ces frameworks est utilisé pour résoudre différents problèmes qui se présentent dans le processus de développement de logiciels. Ils sont utilisés dans de nombreuses industries, y compris le développement web, la science des données, et plus encore. Nous allons examiner les catégories plus larges de frameworks de développement web avec des exemples spécifiques.

I.3.3.1. Frameworks de développement web back-end

Les frameworks back-end (frameworks côté serveur) fournissent des fonctionnalités génériques qui peuvent être assemblées ou construites pour faciliter le développement web côté serveur. Comme framework de développement d'applications web back-end nous pouvons citer : Spring Boot, ASP.NET, ASP.NET Core, Ruby on Rails, Laravel, CodeIgniter, CakePHP.

I.3.3.2. Frameworks de développement web front-end

Alors que les frameworks web back-end sont exécutés sur un serveur, les frameworks front-end sont exécutés dans le navigateur de l'utilisateur. Ils permettent aux développeurs web de concevoir ce que les utilisateurs voient sur le site web, par exemple la gestion des requêtes AJAX, la définition des structures de fichiers et le style des composants du site web. Comme il s'agit de frameworks, ils offrent aux développeurs web une base de développement tout en leur permettant de contrôler le résultat final. Les frameworks d'applications web front-end les plus courants sont : Angular, REACT JS, Vue JS.

Les frameworks réduisent considérablement le temps et l'énergie consacrés au développement de logiciels. Tous les détails de bas niveau sont déjà inclus. Cela permet au développeur de consacrer plus de temps à l'écriture du code spécifique au projet, et non à la gestion des fonctions banales et répétitives du framework. Ils rendent également le code propre et facilement adaptable en suivant la convention de codage du framework. Enfin, les frameworks permettent de développer le projet rapidement, en réduisant le temps de programmation ainsi que les coûts associés.

Cependant, les frameworks bien qu'étant une solution universelle, ils comportent certaines restrictions. Vous ne pouvez pas faire tout et n'importe quoi avec un seul framework. De plus Ils ne conviennent pas aux petits projets, car la mise en place du cadre prend plus de temps que le codage personnalisé du projet. Un autre détail est que en tant que nouvel utilisateur du framework, le développeur doit prendre le temps de comprendre comment l'utiliser correctement avant d'être productif. Enfin, les bugs ou les problèmes de sécurité d'un framework peuvent affecter toutes les applications construites sur ce framework.

I.3.4. Sécurité dans les applications web

Les politiques de contrôle d'accès sont mises en place dans le but de définir des règles pour l'accès aux informations qui circulent dans l'entreprise. Les applications web adoptent ces politiques dans le but garantir une certaine sécurité dans la manipulation des données sensibles telles que les données de utilisateurs par exemple. Dans cette section nous présenterons la politique de contrôle d'accès RBAC (Role Based Access Control) qui est une politique très utilisée sur le web.

Historiquement, le contrôle d'accès basé sur les rôles (RBAC) est apparu rapidement dans les années 90 comme une technologie pour gérer et renforcer la sécurité des systèmes à grande échelle dans les entreprises [4]. Celui-ci a été mis en place au sein des serveurs web dans le but de remplacer les politiques de contrôle d'accès qui étaient principalement basées sur l'identité des l'utilisateurs de manière

individuelle. Ainsi on a pu constater que lorsque l'on fournit de manière sécurisée les rôles des utilisateurs individuels [41], les serveurs web peuvent se baser sur ces rôles pour contrôler l'accès aux ressources. De cette facon, un couplage entre le web et RBAC a donc eu un impact considérable sur le déploiement d'une sécurité efficace dans des systèmes à grande échelle [4].

Le fonctionnement de base du modèle RBAC [1] suit la logique selon laquelle les autorisations ou permissions sont associées à des rôles et que les utilisateurs sont affectés aux rôles appropriés. Cela a pour effet de simplifier considérablement toutes les contraintes liées à la gestion de la sécurité. Ainsi RBAC est une alternative aux contrôles d'accès discrétionnaires et obligatoires traditionnels ; il garantit que seuls les utilisateurs autorisés ont accès à certaines données ou ressources. Il prend également en charge trois principes de sécurité bien connus : le masquage des informations, le moindre privilège et la séparation des tâches. Un rôle est une construction sémantique qui constitue la base de la politique de contrôle d'accès. Avec RBAC, les administrateurs système peuvent créer des rôles, accorder des autorisations à ces rôles, puis affecter des utilisateurs à ces rôles en fonction de leurs besoins spécifiques.

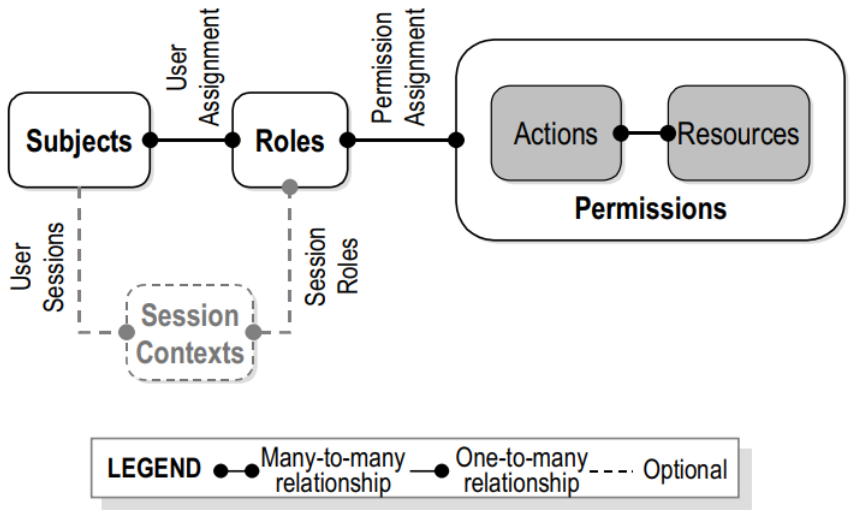


FIGURE 6 – *Cœur du modèle RBAC (source [3])*

1. Une famille de modèles de contrôle d'accès basés sur les rôles. RBAC0, le modèle de base en bas, est l'exigence minimale pour un système RBAC. Les modèles avancés RBAC1 et RBAC2 comprennent RBAC0 mais RBAC1 ajoute des hiérarchies de rôles, tandis que RBAC2 ajoute des contraintes. Le modèle consolidé, RBAC3, comprend RBAC1 et RBAC2 et, par transitivité, RBAC0 (source [4]).

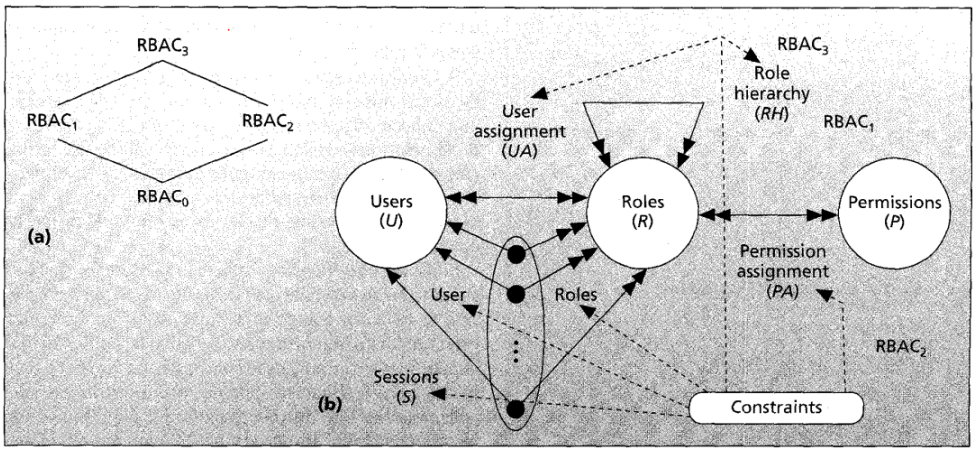


FIGURE 7 – *Une famille de modèles de contrôle d'accès basés sur les rôles. (source [4])*

I.4. Conclusion

En somme, l'objectif principal de la discipline du BPM est l'automatisation des processus métier. Pour ce faire, cette discipline met à la portée des concepteurs de systèmes d'information, un ensemble d'outils et techniques visant à concevoir, déployer, exécuter et monitorer les processus des entreprises. De cette même façon les applications web sont des systèmes d'information très utilisés aujourd'hui. Des milliers d'utilisateurs traitent et partagent l'information de par le monde ce qui fait de l'industrie du développement web une grande part de marché qui nécessite néanmoins des ajustements innovatifs dans le but de faire grandir ce domaine d'avantage.

Dans ce chapitre nous avons posés les notions de base sur les domaines du BPM et du développement web. Dans le chapitre suivant nous présenterons comment ces deux disciplines se rencontrent.

II

CHAPITRE

ÉTAT DE L'ART SUR LA SPÉCIFICATION DES PROCESSUS MÉTIER POUVANT S'EXÉCUTER AU SEIN D'UNE APPLICATION WEB

SOMMAIRE

II.1 - Introduction 26
II.2 - Ingénierie Dirigée par les Modèles (IDM) 27
II.3 - Business Process Management et industrie du Développement web . 30
II.4 - Vision métier pour la modélisation des applications web 34
II.5 - Conclusion 41

II.1. Introduction

Dans le chapitre précédent, nous avons introduit la discipline du Business Process Management ainsi que celle du développement web. Principalement nous avons présenté les notions de base de la discipline BPM ainsi que les différents paradigmes et techniques qui interviennent dans la conception et la mise en œuvre des applications web. Dans ce chapitre nous allons présenter comment ces deux disciplines se rencontrent, les enjeux et les avantages de l'emploi des techniques du BPM et de l'IDM dans la conception et la réalisation des applications web. Dans un premier temps, nous allons faire un rappel sur les concepts de l'IDM. Ainsi ce chapitre sera subdivisé de la manière suivante : la section II.2 présente les concepts de l'IDM qui sont utilisés dans la plupart des approches de l'état de l'art. La section II.3 présente le Business Process Management et l'industrie du Développement web (Le motivations et les enjeux), la section II.3.1 présente les principales

approches traditionnelles d'ingénierie web pour la modélisation des applications web, nous établissons une classification de celles-ci puis nous listons leurs insuffisances. Ensuite la section II.4 introduit une vision métier pour la conception des applications web en définissant la notion d'*application web basée sur les processus métier* et en donnant le rôle de la prise en compte des processus métier pendant la première phase d'analyse (le recueil des besoins). Dans la section II.4.3 nous présentons une vue d'ensemble sur les approches de gestion des processus métier dans les applications web, nous listons quelques limites liées à ces approches avant d'introduire notre solution.

II.2. Ingénierie Dirigée par les Modèles (IDM)

Le but de cette section est d'introduire l'IDM et son importance pour mettre en œuvre notre approche.

II.2.1. Définition et principe

Le Génie Logiciel est passé par plusieurs phases déterminantes dans le but de maîtriser la complexité grandissante du développement des logiciels. Ainsi, des avancées technologiques majeures ont vu le jour, partant des technologies procédurales jusqu'à celles des composants en passant par les technologies objets. Dans cette évolution, le modèle a toujours pris une forme passive (c'est-à-dire contemplative) en n'étant qu'un moyen d'échange d'idée lors de la phase de conception [42]. Pour améliorer les modèles, certains se sont dit : et si le modèle devenait productif ! Et comment faire ? Et pourquoi ? L'IDM donne des réponses à ces questions.

Dès la création de la première machine programmable, l'homme a créé des langages lui permettant d'interagir avec la machine et de lui transmettre des données. Le langage machine étant une suite des zéros et des uns, un procédé permettant de passer du langage naturel au langage machine est apparu nécessaire. L'un des premiers procédés performants fut la compilation. Il s'agit d'un processus de production permettant de passer d'un langage déclaratif (sous ensemble non ambigü du langage naturel exprimant les ordres à réaliser) au langage machine [43]. Aujourd'hui, on trouve des langages, comme le C, munis de compilateurs permettant de transformer le programme d'entrée en un code exécutable par la machine. Une autre approche consiste à interpréter le code par un programme appelé « Interpréteur » qui est, souvent, écrit dans un langage compilable. Par exemple le langage Python avec son interpréteur écrit en C. Le code est alors transformé en une liste

d'ordres au moment de l'exécution, mais pour des raisons de performance le résultat de cette transformation peut être stocké afin d'être rejoué si l'on exécute à nouveau le même programme.

L'apparition des langages de programmation avec leur compilateur ou leur interpréteur vise à simplifier la construction des programmes [43]. Cela a constitué une étape importante pour rendre la construction des logiciels plus accessible.

Ces dernières décennies, nous assistons à l'émergence d'une « nouvelle » forme d'ingénierie des logiciels, centrée sur le modèle, et appelée Ingénierie Dirigée par les Modèles (IDM) [44, 45, 46]. L'IDM offre un cadre méthodologique et technologique prometteur permettant d'unifier différentes façons de faire en un processus homogène et de favoriser l'étude des différents aspects du système.

L'IDM est une forme d'ingénierie générative qui aboutit au code du système décrit à haut niveau d'abstraction [43]. Elle se focalise sur une modélisation abstraite du système réel en un modèle, et un passage automatique de ce modèle à un langage compréhensible par la machine. Ce passage automatique, fait par la machine, rend le modèle lui-même accessible par la machine et est équivalent à la compilation. En parallèle avec les approches basées sur les langages classiques, l'IDM présente une étape majeure pour rendre la construction des logiciels encore plus accessible.

Dans l'approche IDM, le modèle est au cœur du processus de développement logiciel. L'utilisation intensive et systématique des modèles dans une telle approche implique que les différents modèles soient des entités intelligibles par la machine. En parallèle avec le "tout est objet", l'IDM consiste à dire que "tout est modèle" [47].

À la différence des approches classiques fondées sur les langages technologiques, l'IDM permet de couvrir tous les niveaux de conception d'une application web. Les techniques de passage d'un niveau à un autre, quant à elles, restent communes entre les différentes approches. Enfin, les techniques de passage entre les différentes approches se limitent à la transcription (ou génération de code).

Dans une approche IDM, il faut faire la distinction entre la façon dont on exprime les besoins (ici les exigences métier), le but de cette expression (ici le code de l'application) et enfin la méthodologie adoptée pour passer de l'un à l'autre (ici le processus de production). L'expression des besoins ne doit pas être polluée par les techniques de passage de celles-ci au code final et doit se formuler dans un langage métier situé à un haut niveau d'abstraction.

La formalisation des besoins métier (c'est-à-dire le passage du niveau informel au niveau formalisé), la virtualisation des besoins métier (c'est-à-dire le passage

du niveau formalisé au niveau technique) et enfin la production du code de l'application (c'est-à-dire le passage du niveau technique aux niveaux technologiques) peuvent être faites d'une manière automatique par transformations de modèles [46]. Ceci entraîne une succession de transformations de modèles, chaque transformation permettant de faire passer d'un niveau d'abstraction au niveau d'abstraction inférieur, jusqu'à atteindre le niveau voulu. L'enchaînement et l'organisation de ces transformations et très important et constitue un processus appelé processus de production de logiciels.

Si le processus de production logiciel n'est pas automatisé, il est alors réalisé par intervention d'un spécialiste : expert, analyste, développeur, intégrateur, etc. On comprend alors la force de l'IDM si l'expression du cahier des charges se fait par un modèle pouvant être automatiquement transformé et si en plus ce cahier des charges utilise un modèle exprimé par un DSL (Domain Specific Language) spécifique au métier ciblé permettant au client de lui-même exprimer son besoin sans intermédiaire.

II.2.2. Principaux concepts de l'IDM

Dans l'approche IDM, nous pouvons identifier les modèles et les transformations de modèles. Les transformations de modèle sont aussi des modèles. Ce qui confirme que dans une approche IDM tout est modèle. Les transformations de modèle sont souvent accompagnées par un moteur de transformations permettant leur exécution. Les transformations de modèles permettent de rendre les modèles intelligibles par la machine. Dans cette section nous détaillons les différents types de modèles et de transformations de modèles.

II.2.2.1. Les modèles

Selon la décomposition de l'OMG [1] nous pouvons trouver trois types de modèles ;

— le *modèle* qui peut être défini comme un ensemble d'informations ou de connaissances en relation avec un domaine donné. Cet ensemble représente un point de vue d'un système réel à un niveau abstrait. Un modèle peut être vu comme une simplification du système réel permettant de répondre à toutes les questions sous-jacentes à un point de vue donné du système. Par exemple le modèle *LSAWfP* (A Language for the Specification of Administrative Workflow Processes) est un modèle qui permet de décrire les processus métier.

1. OMG (Object Management Group) https ://www.omg.org

— le *métamodèle* est un modèle permettant de décrire le modèle. On dit, dans ce cas, qu'un modèle est conforme à un métamodèle. En parallèle avec la notion de langage informatique, le métamodèle est la grammaire et le modèle est le programme. Par exemple l'ensemble des définitions des concepts de *LSAWfP* forme un métamodèle.
— le *métamétamodèle* est un modèle permettant de décrire les métamodèles. C'est donc aussi un métamodèle. Afin d'éviter d'avoir un autre métamodèle pour décrire le métamétamodèle, ce dernier se décrit avec lui-même. Dans le parallèle avec la notion de langage, le métamétamodèle est au métamodèle ce que la description BNF (Backus Naur Form) est à une grammaire, le métamodèle est au modèle ce que la grammaire est à un langage.

II.2.2.2. Les transformations de modèles

Les transformations de modèles permettent de décrire comment modifier ou construire d'autres modèles en se basant sur un ensemble de modèles d'entrée. Cette description est faite à travers des règles. En IDM les transformations de modèles peuvent être des transformations de modèle vers modèles ou des générations de code ;

— La *transformation de modèle vers modèles* permet, d'une manière générale, de passer d'un espace de modélisation à un autre selon des règles de transformation. Ces règles décrivent comment les concepts des métamodèles sources se traduisent en concepts des métamodèles de destination ;
— La *génération de code* est une transformation de modèle dont la destination est du texte. Elle est aussi appelée transformation de modèles vers texte. Il n'y a aucune restriction sur le texte généré, qui peut être aussi bien du XML, du Java ou du Python.

Dans cette section nous avons définit l'IDM et présenté ses concepts de base ainsi que ses méthodologies. Dans la section suivante, nous présentons la nécessité d'intégrer la gestion des processus métier au sein des applications web.

II.3. Business Process Management et industrie du Développement web

Une grande partie des activités que nous menons au sein des applications web peut être décrite en termes de processus. En effet la liste est très exhaustive. Que ce soit lorsqu'on crée un compte sur un réseau social, lorsqu'on fait un achat sur un site de e-commerce, lorsqu'on dépose un dossier en ligne pour un recrutement,

lorsqu'on crée une publication sur un réseau social tel que Facebook, Twitter ou Instagram, lorsqu'on soumet une réclamation dans une agence, lorsqu'on suit une procédure de préinscription dans une université en ligne ou encore lorsqu'on soumet un article dans une revue scientifique ; nous sommes emmenés à exécuter des processus. En effet, chacune de ces activités concourent à l'atteinte d'un objectif spécifique (Une inscription, un dépôt de dossier, la publication d'un article ...) et peut être décrites en termes de tâches, d'acteurs, de rôles, de permissions, et de flows de contrôle. Pour aller plus loin, certaines applications web sont typiquement concues pour l'offre des services en ligne et ne font qu'exposer des procédures et activités que des "clients" viennent initier et suivre. C'est le cas par exemple des sites web d'organismes gouvernementaux ou encore des plateformes de e-learning. D'autres applications web ne sont pas typiquement basées sur les processus métier mais, il arrive quand même qu'elles exposent des fonctionnalités et activités qui peuvent être décrites en termes de processus.

Toutefois, les stratégies de développement web traditionnelles se penchent sur une analyse et une conception des systèmes d'information (précisément des applications web) avec pour point focal "*les fonctionnalités*" plutôt que "*les processus*". En utilisant une approche traditionnelle, on implémente les fonctionnalités de manière individuelles, puis l'on assemble celles-ci pour fournir le système final. Les principaux problèmes avec cette approche réside sur plusieurs constats qui ont été établis par le Cutter Consortium [48] :

- seulement 16% des applications répondent aux besoins métier réels,
- les délais de livraison des applications ne sont pas respectés dans 79% des cas,
- les budgets consacrés au développement des applications sont dépassés dans 63% des cas,
- la pauvreté des spécifications fonctionnelles touche 53% des cas,
- la qualité des livrables est mauvaise dans 52%.

La discipline du BPM a déjà fait ses preuves quant à la gestion des processus métier. Elle fournit des méthodes, techniques et outils permettant d'identifier, de découvrir, d'analyser, de reconcevoir, d'exécuter et de surveiller les processus métier afin d'en optimiser les performances [8]. Les techniques du BPM permettent de réduire le temps et le coût de réalisation des applications tout en fournissant des outils d'administration et de surveillance sophistiqués. Ces techniques permettent d'obtenir également des applications répondant aux besoins métiers des clients. C'est dans cette optique que de nombreux travaux se sont penchés sur un ensemble de techniques de BPM visant à intégrer et manager les processus au sein des appli-

cations web. Principalement on peut identifier deux courants de modélisation des applications web [49] : la modélisation majoritairement centrée sur les aspects informatiques ou technologiques de l'application (*approches traditionnelles*) considérée comme la réponse aux besoins métier ; et la modélisation majoritairement centrée sur les besoins métier eux-mêmes [49]. Dans le cadre de ce chapitre, nous allons tout d'abord présenter les approches traditionnelles de modélisation des applications web ainsi que leurs limites, ensuite nous nous concentrerons sur la vision métier pour la modélisation des applications web.

II.3.1. Approches traditionnelles d'ingénierie web pour la modélisation des applications web

Schwinger et al. [50] ont établi une classification ainsi qu'une étude comparative entre les différentes approches de modélisation des applications web. Principalement ces approches sont classées en fonction de leur origine et leurs objectifs.

II.3.1.1. Méthodes orientées données

Les méthodes orientées données sont issues du domaine des systèmes de bases de données ; elles sont principalement basées sur le modèle Entité-Association (EA) enrichi de concepts spécifiques pour la modélisation au niveau hypertexte. L'objectif principal de ces méthodes est la modélisation d'applications web pilotées par des bases de données. Parmi les exemples de méthodes orientées données, citons la méthodologie de gestion des relations (RMM) [51], Hera [52] et le web Modeling Language (WebML) [53].

II.3.1.2. Méthodes orientées hypertexte

Les méthodes orientées hypertexte sont centrées sur le caractère hypertexte des applications web ; elles sont issues principalement du domaine des systèmes hypertexte. Les représentants de ce groupe sont le modèle de conception hypertexte (HDM) [54], qui a été étendu à W2000 [55], et HDM-lite, ou la méthode de conception de sites web (WSDM) [56, 57, 58].

II.3.1.3. Méthodes orientées objet

Les méthodes orientées objet sont basées sur OMT (Object Modeling Technique) pour les toutes premières méthodes ou sur UML. UML est la notation préférée lorsqu'un langage standard de modélisation est choisi. Cette catégorie comprend la méthode de conception hypermédia orientée objet (OOHDM) [59], l'ingénierie web basée sur UML (UWE) [60, 61] , les solutions web orientées objet (OOWS) [62] et la méthode hypermédia orientée objet (OO-H) [63].

II.3.1.4. Méthodes orientées logiciel

Les méthodes orientées logiciel considèrent les applications web principalement du point de vue du développement logiciel traditionnel, en utilisant des techniques qui suivent de près le génie logiciel classique. De bons exemples pour cette catégorie sont web Application Extension (WAE), ou WAE2, sa version améliorée.

La figure 8 présente les approches de modélisation des applications web présentées dans les sections précédentes.

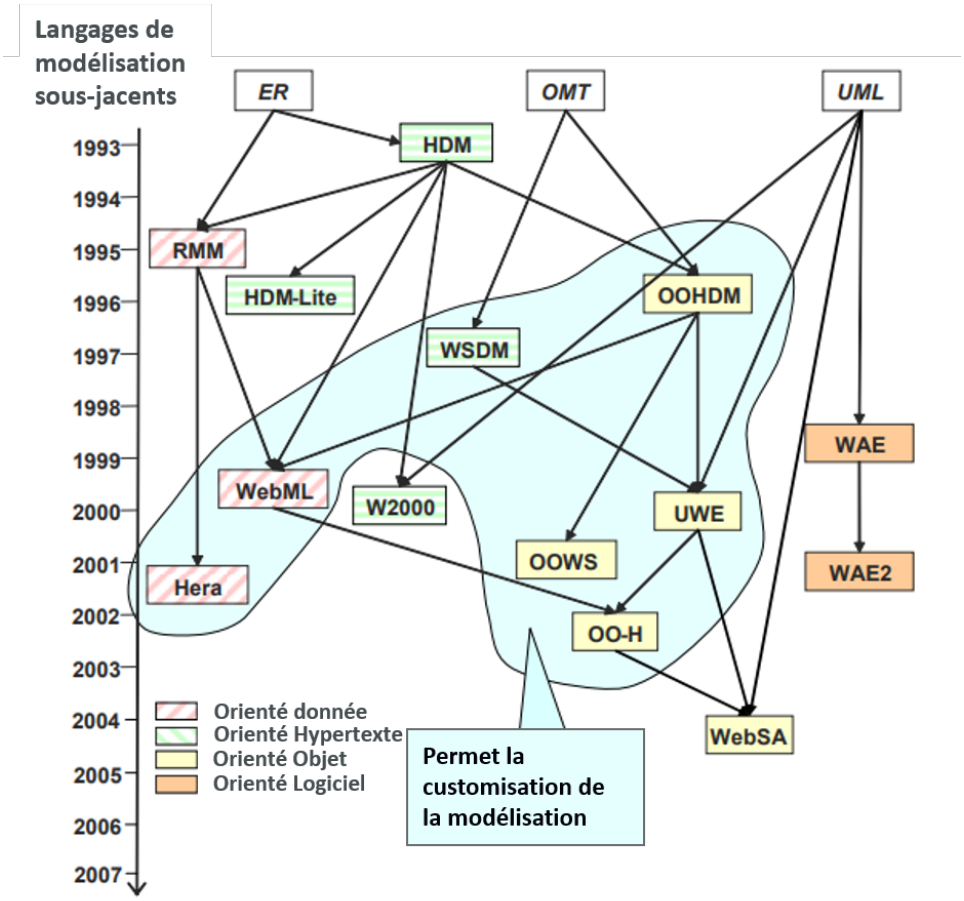


FIGURE 8 – *Vue d'ensemble sur les approches traditionnelles de modélisation des applications web (source [5])*

II.3.2. Synthèse et critique des approches

Les approches traditionnelles de modélisation des applications web présentent un manque d'accessibilité [2].

2. L'accessibilité du métamodèle signifie que le métamodèle est facilement compréhensible par les concepteurs de modèles. Ainsi, plus le métamodèle est accessible, plus il est adopté.

En effet, la finalité des approches traditionnelles va du prototypage des interfaces utilisateurs à la génération partielle ou complète de l'application. Ces approches sont destinées, principalement, aux concepteurs informaticiens et se préoccupent davantage de la solution technique de l'application que du métier des utilisateurs. Aucune de ces approches ne part de la modélisation des entreprises centrée processus métier. En majorité, elles utilisent un métamodèle décrivant l'application sous forme de modèle de navigation entre les pages web rendant le modèle peu accessible aux acteurs métier comme dans le cas de OOWS ou webSpec [64]. D'autres utilisent des diagrammes tels que le diagramme d'activité afin de permettre la description des processus d'implémentation, comme dans le cas de OOHDM ou UWE. Ces processus se situent à un niveau technique décrivant l'implémentation d'une activité métier. Ainsi, nous avons constaté une fusion entre le processus métier et le processus d'implémentation. Cela a comme conséquence la difficulté de décrire des processus métier durables (s'étalant sur une longue période de temps) et complexes, ainsi que la difficulté de décrire les processus de contrôle et de gestion des processus. Dans le cas de WebML, la description des processus métier est implicite et est à un niveau technique d'implémentation. Les processus métier, dans ce cas, sont éclatés sur les différents modèles de navigation. En plus de cela, le concepteur doit gérer les concepts ajoutés automatiquement et décrivant la logique d'exécution des processus métier. Ces détails, selon nous, font partie de l'expertise technique qui doit être rajoutée automatiquement dans les modèles intermédiaires d'un processus de production de logiciels.

II.4. Vision métier pour la modélisation des applications web

II.4.1. Applications web basées sur les processus métier

Torres Victoria et al. [65] définissent les applications web basées sur les processus métier comme étant des applications auxquelles on accède via un navigateur web sur un réseau (Internet ou Intranet) et qui permettent aux utilisateurs d'exécuter un ensemble de tâches clés ou des workflows. Ce type d'application avait préalablement été identifié par Kappel et al [66] comme étant des applications basées sur les tâches. En effet, ce type d'application possède des caractéristiques spécifiques parmi lesquelles :

— *un processus bien défini*. Lorsque les utilisateurs accèdent à une application web basées sur les processus métier, il leur est imposé quelques restrictions. Ils naviguent de manière contrôlée dans l'application et n'ont accès qu'au

contenu et aux fonctionnalités nécessaires selon la définition du processus métier.

— *la coopération homme-système*. Les processus d'entreprise tendent à franchir les frontières des organisations et à intégrer différentes ressources et services informatiques [67]. En outre, l'automatisation complète d'un processus n'est pas toujours possible ou souhaitable [68]. Ainsi, la participation humaine ne peut pas être négligée et des conseils doivent être fournis pour rendre la participation humaine aussi facile que possible.

II.4.2. Rôle des processus métier dans l'analyse de besoins

L'inclusion d'un processus métier dans toute approche de modélisation web affecte chaque étape du développement, de l'analyse des besoins à la mise en œuvre. En ce qui concerne l'analyse des besoins, dont l'objectif est l'élicitation, la spécification et la validation des besoins des utilisateurs et des clients, cette activité comprend la détection des exigences fonctionnelles et non fonctionnelles, qui peuvent toutes deux être affectées par des préoccupations liées aux processus.

Bien qu'il n'existe pas de processus normalisé pour l'analyse des besoins, les meilleures pratiques en matière de développement d'applications logicielles générales fournissent un ensemble de techniques. Une étude comparative menée par Escalona et Koch en 2003 [69] sur les techniques d'ingénierie des exigences pour le développement d'applications web a montré que la modélisation des cas d'utilisation est la technique la plus populaire proposée pour la spécification des exigences tandis que l'entretien (interview) est la technique la plus utilisée pour la capture de ces exigences. Ces résultats ne sont pas surprenants ; le développement traditionnel de logiciels fait appel à l'entretien comme procédure intuitive et largement utilisée pour guider une "conversation avec un but" [70], et les cas d'utilisation sont une approche bien connue pour la représentation graphique et la description des exigences, suggérée par Jacobson et al [71]. La modélisation des cas d'utilisation est un formalisme puissant pour exprimer les exigences fonctionnelles d'applications à forte intensité de travail, web ou non web.

Un tel modèle de cas d'utilisation comprend généralement un diagramme de cas d'utilisation qui est habituellement suffisant pour décrire les fonctionnalités des systèmes simples, tels que les systèmes d'information web. En revanche, les applications web comprenant des processus métier nécessitent une description plus détaillée des séquences d'actions complexes intervenants au sein de celles-ci. De l'analyse des flux de contrôle interne de chaque cas d'utilisation défini on peut noter que certains flux sont triviaux du point de vue de l'entreprise, car ils expriment

uniquement des activités de navigation. En revanche, d'autres, impliquent un flux de contrôle complexe, et nécessitent des raffinements supplémentaires. Il est donc important de capturer et de modéliser les aspects métier avec les outils appropriés tels que des langages de modélisation des processus métier.

II.4.3. Vue d'ensemble sur les approches pour la gestion des processus métier dans les applications web

II.4.3.1. An empirical study of rules for mapping BPMN models to graphical user interfaces

Eduardo Diaz et al. [72] ont mené une étude très récente la génération d'interfaces utilisateur graphiques (GUI) à partir de modèles BPMN. Ils analysent douze projets BPMN et en les comparant avec leurs interfaces graphiques associées puis fournissent un ensemble de règles pour mapper les modèles BPMN aux interfaces graphiques. Cet article apporte trois contributions principales : une validation empirique de ces règles de cartographie ; une classification de leurs alternatives pour identifier celles qui soutiennent la convivialité ; et la validation de cette classification. Cette étude fournit aux analystes des conseils efficaces et efficients sur la façon d'appliquer des règles de transformation de manière cohérente et d'alimenter un logiciel pour la transformation semi-automatique des modèles BPMN dans leurs interfaces graphiques correspondantes.

II.4.3.2. Bizzy v2

Dans le travail [65] Victoria Torres et al. présentent une approche complète pour le développement d'applications web basées sur les processus métier. Cette approche s'étend de la phase de modélisation (la phase où le système est représenté en termes d'un ensemble de modèles) jusqu'à la phase de génération (la phase qui applique un ensemble de règles de transformation pour obtenir les artefacts exécutables). Au niveau de la modélisation, ils définissent un ensemble d'abstractions qui représentent les propriétés de navigation trouvées lors de l'exécution du processus métier. Ils modifient également l'architecture des applications web générées afin de gérer correctement les processus métier. En conséquence, ils introduisent un moteur de processus dans l'architecture de ces systèmes. Ce moteur de processus permet la construction de modèles de navigation plus légers, où le flux de processus est maintenu à l'intérieur de la définition de processus. Sur la base du MDE, ils définissent un ensemble de transformations de modèle pour obtenir : des définitions de processus exécutables exprimées dans le langage WS-BPEL ; et l'ensemble des fichiers (.java, .html et .page) nécessaires au déploiement d'une application en

termes de framework web, plus précisément le framework web Tapestry. Enfin, ils développent un outil (l'outil Bizzy) qui met en œuvre les idées présentées dans leur travail. L'outil a été développé en appliquant les dernières tendances dans le domaine IDM. Il a été construit à l'aide d'outils inclus dans l'environnement de développement Eclipse (le projet Eclipse Modeling et le projet SOA Tool Platform) et l'outil Java BPMN2BPEL.

II.4.3.3. WebRatio BPM

Dans le but de remonter en abstraction, l'outil WebRatio BPM [73] a proposé une méthodologie associant BPMN et WebML pour rendre l'application plus accessible aux acteurs métier. À partir d'un modèle BPMN, l'outil génère un modèle WebML encombré par les métadonnées décrivant implicitement les processus métier et leur exécution. L'application générée par WebRatio est équivalente à une interface utilisateur issue d'une solution BPM. Ainsi les processus métier sont simplement orchestrés.

II.4.3.4. An Artifact-centric Approach to Generating webbased Business Process Driven User Interfaces

Sira et al dans [74] proposent une approche permettant la génération partielle des interfaces utilisateurs. Cette approche se base sur un modèle d'artefact, un modèle de processus métier centré sur les données (artefacts) et enfin les services. Les processus métier dans ce cas décrivent le cycle de vie de chaque artefact.

II.4.3.5. User-Centered Design and Business Process Modeling : Cross Road in Rapid Prototyping Tools

Le travail fait dans [75] propose une approche de prototypage rapide d'interface utilisateur exécutable. À partir d'un modèle de processus métier centré sur les tâches WBM d'IBM, l'outil génère un modèle permettant de décrire l'interface utilisateur. Ce modèle est, ensuite, complété puis transformé en un modèle XML exécutable de l'interface utilisateur. Cette interface est enfin connectée à un noyau fonctionnel permettant une exécution des processus métier centrée sur les tâches. Le modèle métier d'entrée n'est pas obligatoire. Ainsi, le concepteur peut décrire directement son interface en utilisant le métamodèle intermédiaire. Ce dernier permet une description centrée sur l'utilisateur et ne se préoccupe pas des processus métier.

II.4.3.6. Bizzy v1

La volonté de remonter en abstraction a été formulée et la nécessité d'impliquer les acteurs métier dans la phase de conception a été prouvée et adoptée par plusieurs approches. Nous citons, par exemple, le travail fait par [76] qui rajoute la possibilité de modéliser des processus métier centrés sur les tâches, en utilisant la notation BPMN, pour générer un modèle OOWS situé à un niveau technique. Le modèle obtenu doit être complété par le designer ce qui permettra, de générer le code de l'interface utilisateur de l'application. Le modèle BPMN, quant à lui, est transformé vers une définition de processus exécutable WS-BPEL [77]. Dans cette approche, l'application générée permet à l'utilisateur d'exécuter une liste de tâches décrites dans le modèle des processus métier.

II.4.3.7. BonitaSoft

Parmi les solutions BPM distinguées par leur maturité sur le marché, nous citons BonitaSoft [78] qui est un outil Open-source qui propose un éditeur de processus métier respectant le standard BPMN 2.0. Cet outil propose un éditeur convivial de modélisation de processus métier BPMN, un moteur de workflow permettant d'exécuter les processus de l'entreprise et une application web proche d'une application mail pour l'interaction et le contrôle des processus. Ainsi les processus sont interprétés par l'outil et orchestrés par l'application web fournie. Dans ce processus de développement, l'utilisateur modélise ses processus métier, les connecte à son système d'information à travers des connecteurs puis exécute son application.

II.4.3.8. Node-RED

Node-RED [79] est un outil de programmation permettant de relier des dispositifs matériels, des API et des services en ligne de manière nouvelle et intéressante. Node-RED fournit un éditeur de flux basé sur un navigateur qui facilite l'assemblage de flux en utilisant la large gamme de nœuds de la palette. Le point de départ pour la création d'une application sur Node-RED est la création puis l'assemblage des flux décrivant ainsi la logique métier de l'application. Les flux sont ensuite déployés en un seul clic. Le moteur d'exécution léger est construit sur Node.js, tirant pleinement parti de son modèle événementiel et non bloquant. Il est donc idéal pour fonctionner à la périphérie du réseau, sur du matériel à faible coût tel que le Raspberry Pi, ainsi que dans le Cloud.

II.4.3.9. Camunda BPM

Camunda est un outil de modélisation orientée processus métier qui prend en charge la conception, l'automatisation et l'exécution des processus métier à l'aide

de diagrammes BPMN (Business Process Model and Notation) 2.0 et DMN (Decision Model and Notation) au sein de divers types d'application parmi lesquelles des applications web.

La mission de Camunda est de rendre la gestion des processus métier omniprésente en offrant des solutions que les professionnels et les développeurs peuvent utiliser. Avec cet outil, il est possible de gérer les aspects non techniques tels que l'analyse, la modélisation et l'optimisation des processus, ainsi que les aspects techniques de la création d'un support logiciel pour les processus. Son but principal est d'implémenter le BPMN 2.0.

Les principaux éléments de Camunda et les rôles des utilisateurs finaux sont représentés dans la figure 9.

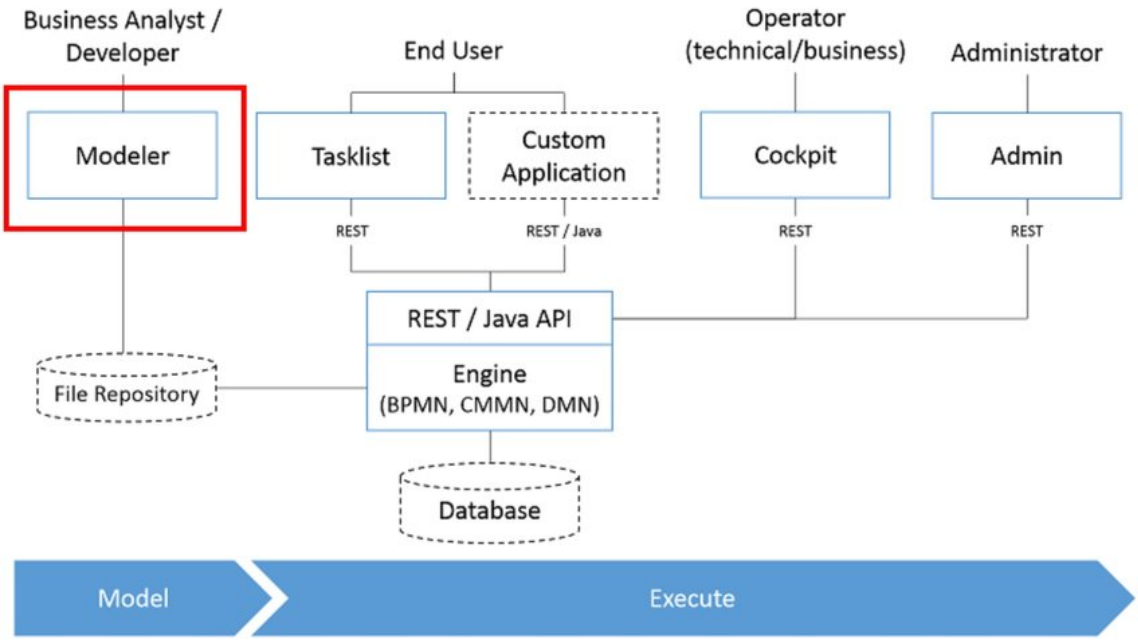


FIGURE 9 – *Composants principaux de Camunda BPM*

II.4.3.10. ASSEMBLE : un outil de développement des processus métier collaboratifs

ASSEMBLE [6] est un outil pour le développement collaboratif de processus métier dans le cloud. ASSEMBLE met en œuvre une approche de mappage des services [80] qui utilise les informations sur les attributs, la structure et la sémantique des opérations de service des processus métier existants dans un domaine donné pour aider une organisation utilisatrice à créer et à déployer ses propres processus dans le cloud. ASSEMBLE génère également des BP exécutables en langage standard BPEL (Business Process Execution Langage), qui peuvent être déployés sur n'importe quel moteur d'exécution de processus n'importe où dans le cloud [6].

ASSEMBLE fournit les fonctionnalités suivantes pour le développement collaboratif de BP : (1) spécification des besoins de l'utilisateur, y compris les opérations de service disponibles et les contraintes de BP ; (2) mise en correspondance des opérations de service internes / partenaires d'une organisation utilisatrice avec les BP existants ; (3) personnalisation et raffinement du BP généré ; et (4) génération du code BP exécutable. La figure 10 présente l'architecture globale de ce système.

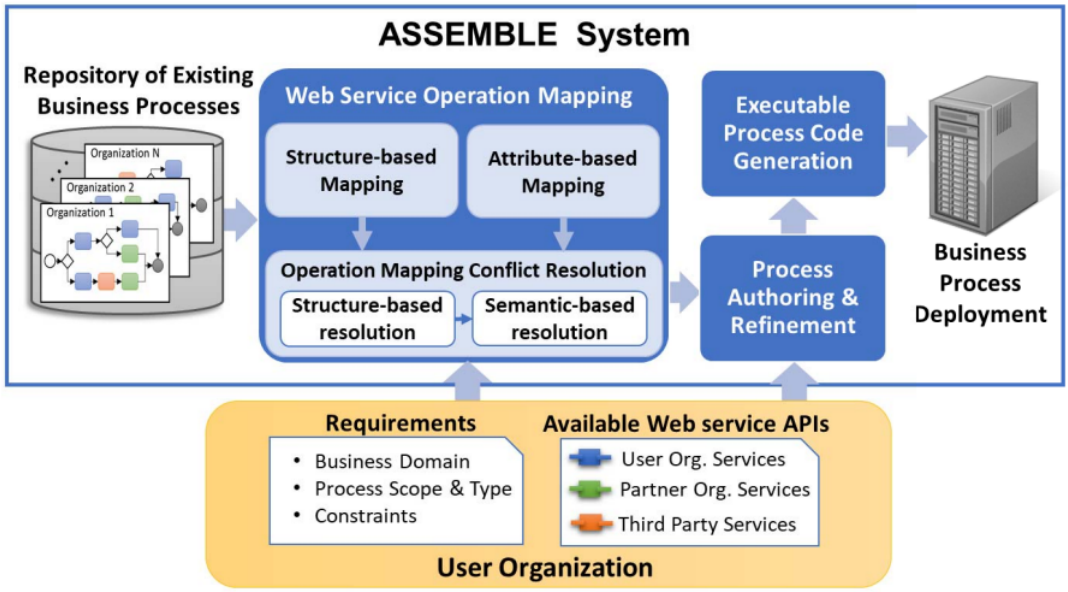


FIGURE 10 – *Architecture globale du système ASSEMBLE (source [6])*

II.4.4. Synthèse et critique des approches

Les approches de gestion des processus métier dans les applications web introduisent la modélisation explicite des processus métier, mais cela reste à un niveau secondaire par rapport à la modélisation de l'application web. En effet, la plupart des solutions BPM de modélisation des processus métier souffrent d'un manque d'expressivité vis à vis de leur utilisation pour la mise en œuvre d'une application web. Dans ce cas, le passage par un modèle intermédiaire comme pour le cas des travaux [65, 73, 75, 76, 6] est obligatoire afin de compléter la modélisation de l'application. Ce modèle intermédiaire constitue le niveau principal de la modélisation. Ainsi les principales approches proposées sont BPMN2OOWS [76], BPMN2BPEL [65], BPMN2WebML [73] ... Dans le cadre de notre travail nous proposons une approche visant à étendre le langage de spécification des processus métier *LSAWfP* proposé par Zekeng et al. [7] pour la construction d'un métamo-

dèle nous permettant de générer des applications web ; l'extension du langage se focalisant naturellement sur l'aspect de navigation uniquement.

II.5. Conclusion

La modélisation des applications web a commencé par la modélisation des systèmes hypertextes et a beaucoup évolué au fil du temps pour couvrir les besoins des nouvelles applications web complexes. Avec l'aspect dynamique croissant des applications web, la notion de système hypertexte est passée d'une forme statique à une forme dynamique. Pour répondre à ce besoin, les modèles de composition [3] et de navigation [4] ont évolués et sont devenus de plus en plus contextualisés. Cette évolution s'est traduite par des modèles de l'application web exprimant les besoins métier de manière implicite (c'est-à-dire intégrés au modèle) et de manière plate (c'est-à-dire répartis sur tous les modèles de l'application web). Pourtant il s'est révelé que ces modèles souffrent d'un manque d'accessibilité, car malgré la tentative de certaines approches comme WebML d'introduire la modélisation des processus métier, ces processus restent inaccessibles par les acteurs métier et sont difficilement exprimables. Ce manque d'accessibilité rend ces approches peu efficaces du point de vue des acteurs métier.

Ces dernières années, avec les avancées technologiques dans le domaine de la génération de code et des transformations de modèles, plusieurs approches ont proposé des méthodologies visant à générer du code pour les applications web en partant d'une modélisation des besoins métier (processus métier ou scénarios). Dans le chapitre suivant, nous décrivons notre approche basée une extension du métamodèle *LSAWfP* pour la génération d'applications web incluant la gestion des processus métier.

3. Le modèle de composition permet de décrire la structure générale du site sous forme de pages web navigables et la structure de celle-ci en termes de contenu

4. Le modèle de navigation ajoute les liens permettant la connexion entre les pages et les unités afin de former une structure hypertexte

III

CHAPITRE

VERS UNE APPROCHE BASÉE SUR L'INGENIERIE DIRIGÉE PAR LES MODÈLES, DE GESTION DES WORKFLOWS DANS UNE APPLICATION WEB

SOMMAIRE

III.1 - Introduction 42
III.2 - Architecture du système 43
III.3 - Extension de *LSAWfP* pour la modélisation des workflows du web : le langage *LSAWfP4WEB* 49
III.4 - Méta-modèle du langage *LSAWfP4WEB* 57
III.5 - Conclusion 57

III.1. Introduction

Dans ce chapitre, nous proposons dans un premier temps une architecture d'application web permettant de faire à la fois de la gestion des processus métier mais aussi du développement classique de fonctionnalités au sein de la même application. En effet, pour la mise en œuvre des fonctionnalités complexes telles que la gestion des commandes sur un site de e-commerce, les préinscriptions dans une université ou les réclamations dans une agence par exemple, le concepteur pourra utiliser une approche basée sur les processus métier tandis que pour la mise en œuvre des fonctionnalités plus simples telles que l'implémentation d'une page de contact, le concepteur pourra utiliser une approche classique. Dans cette marche

vers une approche conciliant les deux courants de pensée présentés dans la revue de la littérature (section II.3), nous proposons une extension du langage *LSAWfP* [7] (A Language for the Specification of Administrative Workflow Processes) nommée *LSAWfP4WEB* pour la spécification des processus métier qui vont s'exécuter au sein d'une application web. Enfin nous proposons un DSEL (Domain Specific Embedded Language) JSON (JavaScript Object Notation) de ce langage pour la création, la sauvegarde et le partage des spécifications.

III.2. Architecture du système

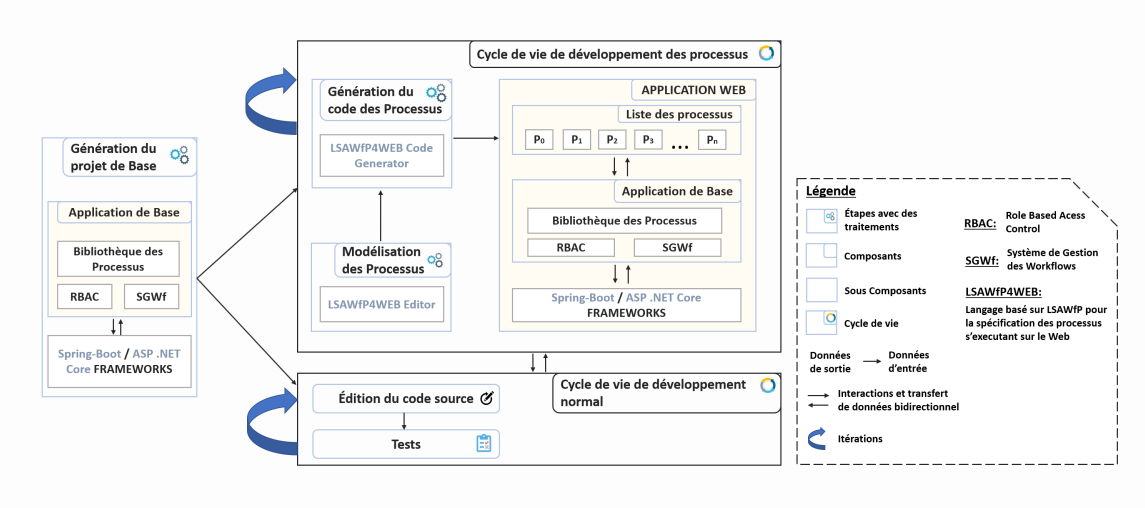


FIGURE 11 – *Architecture du système proposé*

L'architecture de notre système présente trois principaux blocs de base. Le premier bloc décrit les composants d'une *application de base* qui sera générée et utilisée dans les deux autres blocs. Le deuxième et le troisième bloc représentent respectivement le *cycle de vie de développement des processus* et *le cycle de vie de développement classique* au sein de l'application de base. L'idée finale étant de pouvoir illustrer de quelle manière on est capable d'une part de développer des processus et d'autre part de développer des fonctionnalités plus simples au sein de la même application de base. Pour atteindre ces objectifs, nous proposons de générer initialement une application de base avec une structure connue. Nous présenterons en détails cette structure dans la section suivante.

III.2.1. Application de base

L'application de base (figure 12) propose une organisation et repose sur un framework d'applications web tel que Spring-Boot ou ASP .NET Core. Ces frame-

works fournissent des mécanismes et automatismes pour la résolution d'un ensemble de problèmes tels que l'interopérabilité avec les systèmes externes et les bases de données, les problèmes d'extensibilité en offrant la possibilité d'importer des librairies et plugins sur des dépôts distants, des problèmes d'adaptabilité en offrant une pléthore d'outils, de langages ... D'autre part ils sont taillés pour le développement de microservices (les architectures microservices retiennent l'attention de nombreuses entreprises sur le marché actuel).

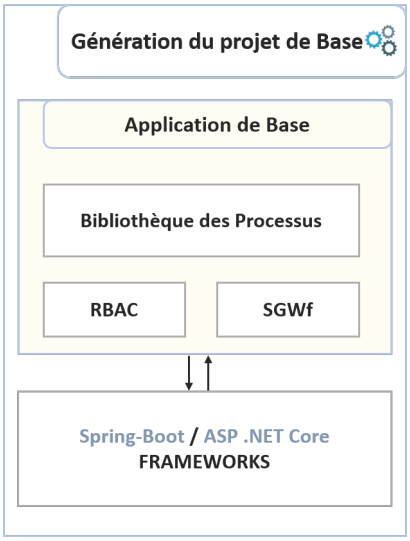


FIGURE 12 – *Application de base*

Pour mieux comprendre les composants constitutifs de l'application de base, nous proposons un diagramme de paquetages illustré sur la figure 13. Ce diagramme présente l'organisation de l'application de base avec quatre principaux paquetages ainsi que des sous-paquetages. Les quatre principaux paquetages sont les paquetages *Spring-Boot Core / ASP .NET Core*, *Process Library*, *RBAC* et *WMS*.

III.2.1.1. Paquetage Spring-Boot Core / ASP .NET Core

Le paquetage Spring-Boot Core (framework Spring-Boot) ou ASP .NET Core (framework ASP .NET Core) constitue la fondation du projet de base (ce paquetage est basé sur l'un des deux frameworks Spring-Boot ou ASP .NET Core). Il fournit une architecture sur laquelle repose le projet. Les paquetages *RBAC* et *WMS* l'utilisent pour l'import des bibliothèques propres aux frameworks Spring-Boot ou ASP .NET et aussi pour l'import des dependances qui auront été téléchargées depuis un dépôt distant.

III.2.1.2. Paquetage RBAC

Il s'agit du paquetage en charge de la sécurité dans le projet de base. Il utilise le paquetage Spring-Boot Core / ASP .NET Core pour l'import des dépendances

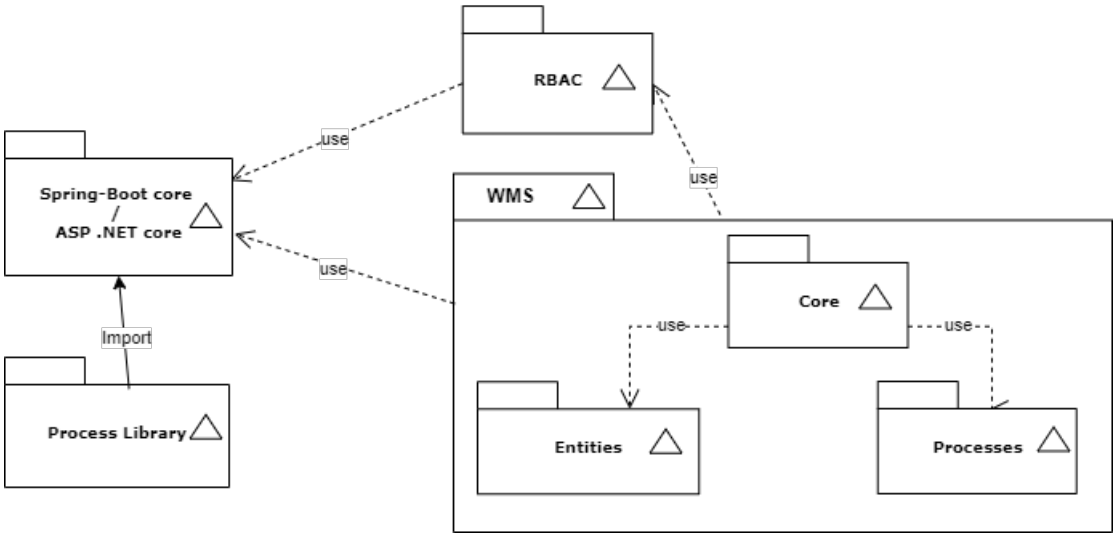


FIGURE 13 – *Paquetages de l'application de base*

liées à la sécurité par exemple l'authentification et la gestion des sessions utilisateur ainsi que le contrôle d'accès aux ressources.

III.2.1.3. Paquetage WMS

Ce paquetage constitue le moteur de gestion des workflows. Il est en charge de l'hébergement et de la gestion des workflows avec les entités que ceux-ci manipulent. Il est formé par trois sous-paquetages principaux (voir figure 13) :

— le sous-paquetage *Core* qui est le cœur du moteur de workflows. Il contient des services permettant de gérer les processus et d'avoir un état global sur ceux-ci (liste des processus gérés dans l'application, nombre d'instances pour chaque processus ...) ;

— le sous-paquetage *Entities* qui contient les entités qui décrivent les données manipulées par les processus au cours de leur exécution ;

— le sous-paquetage *Processes* qui contient uniquement l'ensemble des processus gérés dans l'application par le moteur de workflow. Ces processus sont des sous package ayants eux même une structure particulière qui sera décrite dans la section suivante III.2.1.4.

III.2.1.4. Sous-paquetage Processes du paquetage WMS

Chaque processus contenus dans le paquetage *Processes* possèdent la même architecture comme l'illustre la figure 14. Principalement un processus est un paquetage constitué par les sous-paquetages suivants :

— le sous-paquetage *API* qui représente la couche d'accès au données et de communication avec le processus. Il s'agit de la "porte d'entrée" du pro-

cessus. Cette couche se sert des sous-paquetages *Services* et *Models* pour obtenir respectivement les opérations sur les données et la description de la structure de ces données ;

— le sous-paquetage *Models* contient les modèles décrivant les données manipulées par le processus ;

— le sous-paquetage *Services* fournit des fonctions pour la manipulation des données du processus notamment des *Mappers* pour la conversion d'*entités* à *modèles* et vice versa ; il contient aussi des *Database Initializers* qui sont des scripts d'initialisation de la base de données pour les processus qui nécessitent que des données soient initialement créées pour qu'ils puissent s'exécuter (voir figure 14) ;

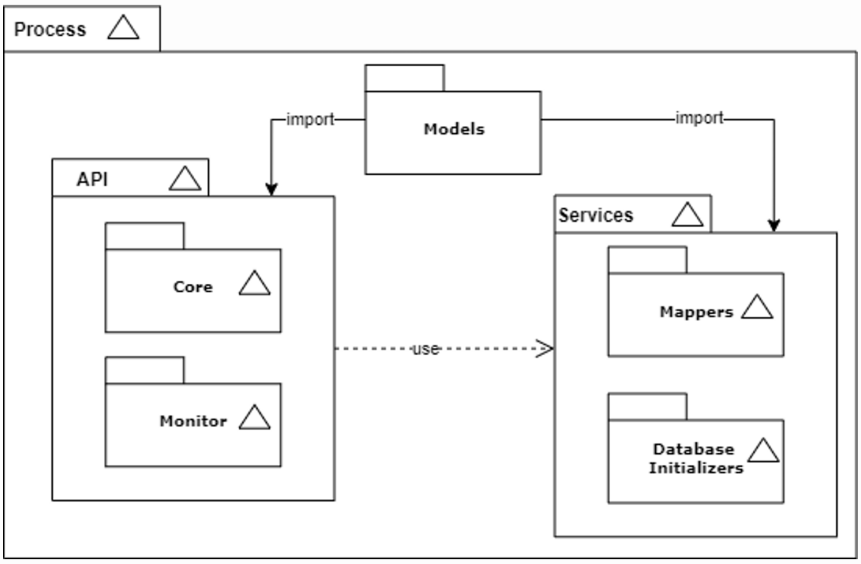


FIGURE 14 – *Paquetages d'un processus dans le moteurs de gestion de workflows*

III.2.1.5. Paquetage Process Library

Ce paquetage contient un ensemble de librairies facilitant la gestion des processus. Principalement, il contient un ensemble de routines (Classes, Annotations, Types, Fonctions ...) pour faciliter la gestion et l'accès aux données des processus. Il s'agit d'un élément clé car il pourra être utilisé par les développeurs pour coder leurs propres processus sans avoir recours à un éditeur d'une part. D'autre part ceux-ci pourrons l'utiliser pour raffiner et améliorer les processus générés s'ils le souhaitent.

Dans la structure du projet de base proposée jusqu'ici, les paquetages *WMS* et *Process Library* permettent non seulement de résoudre le problème de gestion des

workflows mais aussi de le faire d'une manière différente des approches proposées en [65, 73, 75, 76, 6] et aussi les approches des outils *BonitaSoft* et *Camunda BPM* qui prônent une *encapsulation totale* du système de gestion des workflows aux développeurs. De plus les propriétaires des applications web se sentent plus confiants lorsqu'ils ont à l'esprit que les processus de leur application web sont pris en charge au sein de cette même application ; de cette façon, ils auront la possibilité de les suivre et de les raffiner en cas d'erreur sans frais supplémentaires.

À présent que nous avons présenté l'architecture en paquetage de notre application de base tout en précisant le rôle de chaqu'un des composants et en désignant ceux qui seront responsables de la gestion des workflows ; intéressons-nous maintenant aux deux autres principaux composants de notre architecture qui utiliseront le projet de base pour illustrer dans un premier temps *le cycle de vie de développement classique* et dans un second temps *le cycle de vie de développement des processus*

III.2.2. Cycle de vie de développement classique

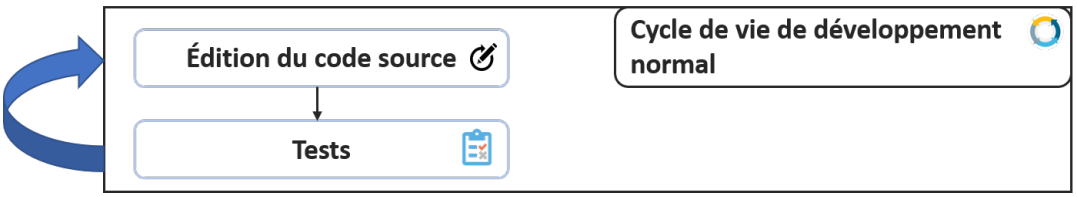


FIGURE 15 – *Cycle de vie de développement classique*

Dans l'architecture proposée, nous mettons exergue le cycle de vie de développement classique. Dans cette partie nous mettons en évidence le fait que si un développeur désire implémenter une fonctionnalité qui ne nécessite pas une approche orientée processus (Cas de l'implémentation d'une page de contact ou d'un simple popup affichant un message de bienvenue à l'utilisateur), il pourra le faire à travers *l'édition du code source* de *l'application de base* qui lui a été fourni. Pendant cette édition il pourra créer de nouveaux paquetages au sein de cette application et pourra effectuer des tests au fur et à mesure du développement des nouvelles fonctionnalités. Les paquetages créés pour l'ajout des nouvelles fonctionnalités pourront accéder en lecture aux données des processus implémentés si la nécessité s'impose.

Cette approche présente comme principal avantage le fait que le développeur ou le concepteur aura libre cours du choix de la méthode employée pour l'ajout de nouvelles fonctionnalités : l'architecture n'impose pas un workflow spécifique pour

l'ajout d'une nouvelle fonctionnalité contrairement aux outils Bonita ou Camunda BPM.

Toutefois ; l'ajout de nouvelles fonctionnalités ne dois pas perturber le système de gestion des processus déjà mis en place (raison pour laquelle le développeur est emmené à créer de nouveaux paquetages différents des paquetages déjà existants et servants à la gestion des processus métier). Le développeur doit avoir conscience du fait qu'il existe une architecture mise en place pour la gestion de processus et qu'il ne doit pas la modifier de manière inappropriée.

C'est de cette façon que l'architecture proposée résout les problèmes des outils comme *Bonita* ou *Camunda* qui sont uniquement taillés pour la gestion des workflows et qui emploient de trop gros moyens pour la résolution de problèmes simples.

À présent que nous avons présenté le cycle de vie de développement classique, intéressons-nous au *cycle de vie développement des processus* qui représente la dernière brique de notre architecture logicielle.

III.2.3. Cycle de vie de développement des processus

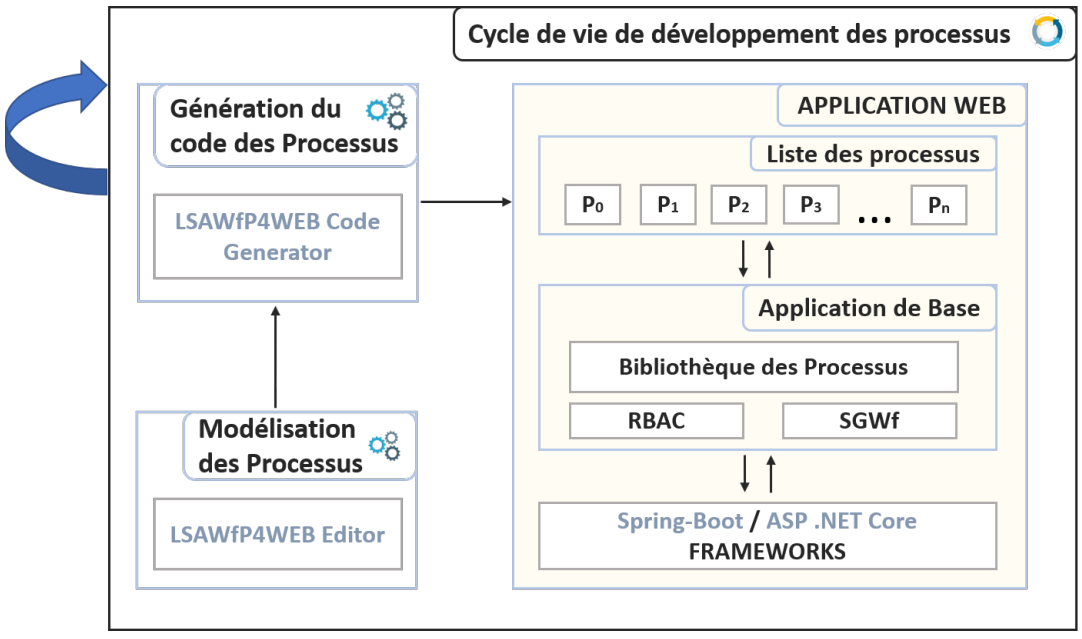


FIGURE 16 – *Cycle de vie de développement des processus*

Toujours dans l'architecture proposée, nous mettons en évidence le cycle de vie de développement des processus. L'idée est de mettre en avant le fait que si un développeur ou un concepteur de systèmes d'information désire développer des fonctionnalités complexes pouvant être décrites au moyen de workflow (le processus de réclamation dans une agence ou encore le processus de commande d'un

produit sur un site de e-commerce), qu'il puisse le faire suivant les principes d'Ingénierie Dirigée par les Modèles (IDM).

Comme l'illustre la figure 16, le développement d'un workflow au sein de notre architecture s'effectue en deux étapes suivant les principes de l'IDM. Dans un premier temps, on modélise le workflow au moyen d'un éditeur et dans un second temps on génère le code du workflow dans un emplacement spécifique du projet de base.

Spécifiquement, pour un workflow donné, on génère dans le paquetage *Processes* (section III.2.1.4), un paquetage ayant la structure présentée à la figure 14. En plus du paquetage du workflow généré, on génère pour chacune des entités manipulées par le workflow, un paquetage qui est sauvegardé dans le paquetage *Entities* (section III.2.1.3). C'est de cette façon que nous sommes en mesure d'inclure la gestion des workflows au sein de notre application web (voir figure 17).

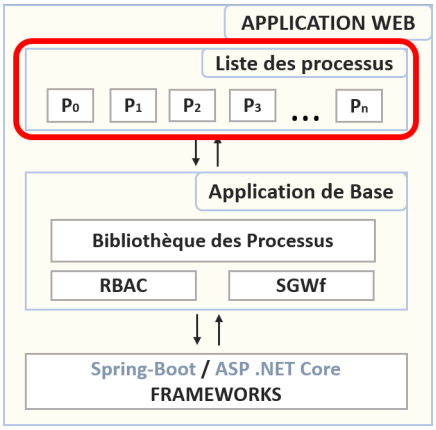


FIGURE 17 – *Présentation des processus déployés au sein de l'application de base*

Dans cette section nous avons présenté la dernière brique de notre architecture. Principalement il était question pour nous de décrire comment notre architecture prend en charge le cycle de vie de développement des processus. Il en ressort clairement que pour pouvoir gérer les processus au sein de notre architecture, nous devons être en mesure de les modéliser en utilisant un langage de workflow ; puis de les spécifier. Ainsi, dans les sections suivantes, nous présentons un langage basé sur *LSAWfP* pour la modélisation des workflows s'exécutant au sein d'une application web.

III.3. Extension de *LSAWfP* pour la modélisation des workflows du web : le langage *LSAWfP4WEB*

III.3.1. Motivation du choix de *LSAWfP* pour la modélisation des workflows du web

Sur le marché actuel, il existe une pléthore de langages de modélisation des workflows parmi lesquels BPMN, BPEL, YAWL, AWGAG etc. Chaque langage ayant des spécificités. Cependant, dans cette pléthore, le langage standard pour la modélisation des workflows est la notation BPMN (Business Process Model and Notation) néanmoins, ce langage bien qu'étant hautement expressif, présente des inconvénients notamment le fait qu'il soit graphique et qu'il ne repose pas sur des bases mathématiques solides [7].

À côté de cette limite, nous avons constaté que la plupart des approches visant à utiliser BPMN pour la modélisation des processus web utilisent un métamodèle intermédiaire pour pouvoir capturer les spécificités du web. Par exemple dans les travaux [65, 76] Victoria Torres se sert du métamodèle OOWS pour capturer les aspect visuels et du métamodèle WS-BPEL pour capturer les aspects liés aux workflows et à la navigation. De même, Marco Brambilla [73], pour le développement de l'outil WebRatio BPM se sert du métamodèle intermédiaire WebML.

Étant donné que le langage LSWAfP est un langage ayant une fondation mathématique solide parce qu'il utilise une variante des grammaires attribuées pour la spécification, et que le modèle de workflow obtenu à partie de *LSAWfP* est capable de prendre en compte à la fois modèle informationnel (avec les attributs de la grammaire), le modèle organisationnel (au moyen des acteurs et de leur accréditations) et le cycle de vie (au moyen des scénarios), il serait très avantageux pour nous de procéder à une extension de celui-ci pour qu'il prenne en compte les spécificités du web (notamment les aspects navigationnels dans un premier temps). Cette extension aura pour effet de proposer un métamodèle basé sur *LSAWfP* pour la modélisation des processus web. Un générateur pourra par la suite se servir des spécifications de ce métamodèle pour générer le code des processus sans que la nécessité de passer par un métamodèle intermédiaire s'impose.

Les motivations étant posées, nous pouvons passer à la présentation du langage *LSAWfP4WEB* qui est une extension du langage *LSAWfP* pour la modélisation des processus du web.

III.3.2. Introduction au langage *LSAWfP4WEB*

Le langage *LSAWfP4WEB* que nous proposons est une *extention* du langage *LSAWfP* (A **L**anguage for the **S**pecification of **A**dministrative **W**ork**f**low **P**rocesses) pour la modélisation des processus du web. Il s'agit d'un langage grammatical simple pour définir les workflows des processus web. Il Met l'accent sur les per-

ceptions potentiellement partielles des différents acteurs sur les processus spécifiés en utilisant le concept de permissions inspiré de RBAC qui est un concept très utilisé sur le web. Le langage *LSAWfP4WEB* fournit un formalisme unique pour la modélisation graphique des processus.

Pour présenter les concepts de ce langage, nous allons nous munir d'un exemple d'application courant qui est *le processus de réclamation*. Il s'agit d'un exemple très utilisé pour présenter la mise en œuvre les processus du web. C'est d'ailleurs l'exemple utilisé par l'outil *Bonita* pour l'initiation à la modélisation des processus web. Nous allons néanmoins le simplifier dans le but de mieux assimiler les étapes de conception.

III.3.3. Présentation du processus de réclamation

Les réclamations désignent des requêtes (sous forme de descriptions textuelles) faites par des clients à des employés d'une agence en vue de l'obtention d'une réponse satisfaisante.

Au sein d'une agence offrant des services aux clients divers, nous savons que s'assurer que les réclamations sont traitées à temps opportun par le(s) employé(s) approprié(s) peut-être la clé de satisfaction du client ; le traitement efficace des réclamations des clients permet aux agences d'accroitre davantage leur clientèle et de se faire plus de bénéfices.

Maintenant que nous avons défini ce que c'est qu'une réclamation et parlé des-en-jeux liés au traitement des réclamations, nous pouvons passer à une description de ce processus.

Voici un aperçu de l'exécution du processus :

— le processus démarre lorsqu'un client soumet une réclamation qui pour notre exemple n'est qu'une simple description textuelle ;
— ensuite, un employé examine la réclamation et fournit une réponse ;
— le client pourra examiner la réponse de l'employé et fournir une note de satisfaction ;
— si la note est supérieure à 3, le client est considéré comme satisfait et le processus se termine ;
— si la note est inférieure ou égale à 3, une tâche supplémentaire sera confiée au manager de l'employé pour travailler avec ce client qui n'est pas vraiment satisfait.
— une fois cette tâche terminée, le processus se termine.

III.3.4. Etapes préliminaires de modélisation du processus de réclamation avec *LSAWfP4WEB*

Pour modéliser le processus de réclamation avec le langage *LSAWfP4WEB*, nous allons commencer par identifier les tâches et les acteurs responsables de ces tâches. Pour le cas d'espèce, nous identifions les acteurs suivants :

— le Client (CL) ;
— l'Employé (EM)
— le Manager de l'employé (MA)

ainsi que les tâches suivante :

— la soumission de la réclamation par le Client (A) ;
— la lecture et l'envoi d'une réponse à la demande du client par l'employé (B) ;
— l'attribution d'une note à le réponse de l'employé par le client (C) ;
— la prise en charge du client par le manager de l'employé (D),

Le tableau I résume l'ensemble des tâches suivi de leur exécutants respectifs.

TABLE I – *Liste exhaustive des tâches du processus de réclamation et leurs exécutants respectifs*

Tâches	**Symboles**	**Acteurs**
Soumission de la réclamation	A	CL
Lecture et l'envoi d'une réponse à la demande du client	B	EM
Attribution d'une note à la réponse de l'employé	C	CL
Prise en charge du client par le manager de l'employé	D	MA

À présent que nous avons décrit et présenté les tâches du processus, passons à la présentation du modèle grammatical que nous proposons.

III.3.5. Modèle de workflow du langage *LSAWfP4WEB*

Dans cette section nous présentons le modèle de workflow du langage *LSAWfP4WEB*, nous utilisons la plupart des concepts du langage *LSAWfP*. Quelques-uns de ces concepts sont pris en compte différemment.

Initialement, le modèle de workflow proposé par le langage *LSAWfP* est un triplet $\mathbb{W}_f = (\mathbb{G}, \mathcal{L}_{P_k}, \mathcal{L}_{\mathcal{A}_k})$ composé par un modèle grammatical de workflows $\mathbb{G}$ qui décrit le workflow étudié, les tâches et leur enchainement sous la forme de scénarios d'exécution ; une liste d'acteurs $\mathcal{L}_{P_k}$ regroupant les acteurs responsables de l'exécution des tâches du processus et enfin une liste d'accréditations $\mathcal{L}_{\mathcal{A}_k}$ désignant les accréditations des acteurs sur les tâches (lecture, écriture, exécution).

Dans le contexte du web, les concepteurs ont libre cours dans la modélisation des acteurs (communément appelés utilisateurs). Étant donné que le processus spécifié va être déployé dans une application web où des acteurs auront été préala-

blement définis, nous n'allons pas prendre en compte la modélisation et la représentation des acteurs dans le modèle *LSAWfP4WEB*. Nous nous mettons dans une configuration où ceux-ci existent déjà et nous nous concentrons sur les permissions que ceux-ci doivent posséder pour lire les données des tâches puis pour exécuter les tâches. La liste des acteur $\mathcal{L}_{P_k}$ n'est donc plus nécessaire.

De plus étant donné que la liste des accréditations $\mathcal{L}_{\mathcal{A}_k}$ associés aux acteurs est inspiré de la gestion des droits utilisateurs dans les systèmes UNIX ce qui n'est pas adapté pour le web, nous allons aussi l'élaguer et la remplacer par un ensemble de permissions qui seront des attributs de chaque tâches du processus.

De toute évidence, le modèle de workflow que nous proposons se réduit à une grammaire attribuée $\mathbb{G}$ possédant les caractéristiques $\mathbb{G} = (\mathcal{S}, \mathcal{P}, \mathcal{A}, Ent)$ où :

- $\mathcal{S}$ est un ensemble fini de *symboles grammaticaux* correspondants aux différentes *tâches* à exécuter dans le processus web. Chacune des tâches possèdent les attributs suivants :
  - *status* qui sauvegarde les informations obtenues après l'exécution de la tâche correspondante.
  - *permissions* qui est un couple de listes (P_r, P_x) où :
    - P_r désigne un ensemble fini de permissions nécessaires à la lecture des données de la tâche S_i par un acteur ;
    - P_x désigne un ensemble fini de permissions nécessaires à l'exécution de la tâche S_i par un acteur ;
  - *group* qui désigne le groupe auquel la tâche appartient. Nous définissons la contrainte selon laquelle pour un processus, les tâches du même groupe sont exécutées par le même acteur.
- $\mathcal{A} \subseteq \mathcal{S}$ est un ensemble fini de symboles grammaticaux particuliers appelés *axiomes* représentants les tâches qui démarrent l'exécution d'un scénario ;
- *Ent* est un ensemble fini d'entités. Une entité E_i est un couple $(nomEntité, typeEntité)$ où *nomEntité* désigne le nom de l'entité qui est unique et *typeEntité* désigne le type de l'entité ;
- un *typeEntité* sert à la génération de formulaires pour la récupération des données d'entrée d'une tâche. Concrètement, un *typeEntité* est soit :
  - un type primitif (chaine de caractère, nombre, ...) ;
  - une liste de types entités ;
  - un type *class* qui est un ensemble fini d'entités
- $\mathcal{P}$ est un ensemble fini de *productions* décorées par les annotations "$\wp$" (est séquentiel à) et "$\|$" (est parallèle à) : ce sont les *règles de précedence* ;
- une production $P = \left(X_0, X_1, \cdots, X_{|P|}, typeEnt\right)$ est de l'une des deux formes :

— $P : X_0 \rightarrow X_1 \fatsemi \ldots \fatsemi X_{|P|} \Rightarrow typeEnt$
— $P : X_0 \rightarrow X_1 \parallel \ldots \parallel X_{|P|} \Rightarrow typeEnt$

Le langage *LSAWfP4WEB* ainsi décrit apporte de nouveaux concepts au langage *LSAWfP* de base pour la prise en compte des *entités*, des *groupes* et des *permissions*.

— Les *entités* décrivent les données manipulées par les processus. Pour le cas du processus de réclamation on a comme exemple d'entité $(desc, String)$ désignant la description textuelle de la réclamation, $(resp, String)$ désignant la réponse de l'employé à la description textuelle, $(note, Integer)$ désignant la note attribuée par le client à la réponse de l'employé.
— Les *groupes* correspondent à un ensemble de tâches effectuées par le même "utilisateur". Concrètement on considère que les tâches du même groupe seront exécutées par le même utilisateur.
— Les *permissions* permettent de sécuriser l'accès aux ressources ainsi que l'exécution des tâches. Le processus doit être déployé au sein d'un système ou des utilisateurs disposent de permissions. Si un utilisateur possède les permissions pour l'exécution d'une tâche, une fois qu'il a exécuté la première tâche du processus, le reste des tâches du processus possédant le même groupe que la première tâche qu'il a effectuée lui seront assignées.

Nous avons ainsi présenté le modèle grammatical de workflows web selon notre approche. À présent nous pouvons terminer la modélisation du processus de réclamation en définissant la grammaire associée.

III.3.6. Etapes finales de modélisation du processus de réclamation avec *LSAWfP4WEB*

Une fois le modèle complètement définit, intéressons-nous à la suite de la modélisation du processus de réclamation.

On distingue deux principaux scénarios d'exécution :

— *scénario 1 :* le client soumet la réclamation (A), l'employé lui fournit une réponse (B), le client attribut une note supérieure à 3 à la réponse de l'employé (C) et le processus se termine ;
— *scénario 2 :* le client soummet la réclamation (A), l'employé lui fournit une réponse (B), le client attribut une note inférieure ou égale à 3 à la réponse de l'employé (C), le manager prend en charge le client (D) et le processus se termine ;

Les artefacts représentatifs [7] de ces scénarios d'exécution sont illustrés dans la figure 18.

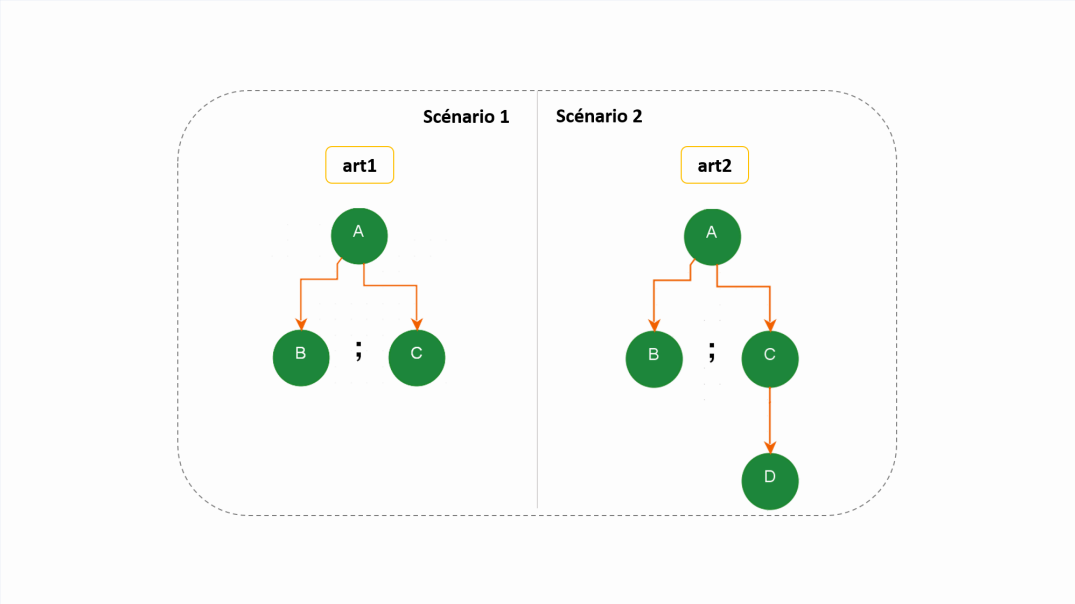


FIGURE 18 – *Artefacts représentatifs du workflow de réclamation*

Une fois les scénarios identifiés, nous pouvons passer à la déduction des permissions associées à la lecture des données et à l'exécution de chaque tâche. Pour un cas précis avec le processus étudié, le client doit avoir la permission de lecture de l'entité description textuelle de la réclamation ($P_r(desc)$) et la permission d'exécuter la tâche soumettre une réclamation ($P_x(A)$) pour pouvoir soumettre une réclamation. Nous raisonnons ainsi pour l'ensemble des tâches du processus et fournissons le récapitulatif dans le tableau II.

TABLE II – *Liste exhaustive des permissions pour l'accès aux données et pour l'exécution de châque tâches*

Tâches	P_r	P_x
La soumission de la réclamation (A)	$\{P_r(desc)\}$	$\{P_x(A)\}$
La lecture et l'envoi d'une réponse à la demande du client (B)	$\{P_r(desc), P_r(resp)\}$	$\{P_x(B)\}$
L'attribution d'une note à la réponse de l'employé (C)	$\{P_r(desc), P_r(resp), P_r(note)\}$	$\{P_x(C)\}$
La prise en charge du client par le Manager de l'employé (D)	$\{P_r(desc), P_r(resp), P_r(note)\}$	$\{P_x(D)\}$

Une fois les permissions définies, nous procédons à présent à l'attribution des groupes aux différentes tâches. Pour notre exemple, le client effectue les tâches A et C donc nous attribuons à ces tâches le groupe G_1. Pour le cas des tâches B et D, nous les attribuons les groupes G_2 et G_3. Les tâches et leurs groupes respectifs sont représentés dans le tableau III.

TABLE III – *Liste des tâches et groupes respectifs*

Tâches	Groupes
A	G_1
B	G_2
C	G_1
D	G_3

Enfin, nous pouvons présenter le modèle grammatical avec les productions et les types entités retournés pour chaque production.

$$P_0 : A \rightarrow B \,;\, C \Rightarrow String \qquad P_1 : B \rightarrow \varepsilon \Rightarrow String$$
$$P_2 : C \rightarrow \varepsilon \Rightarrow Integer \qquad P_3 : C \rightarrow D \Rightarrow Integer$$
$$P_4 : D \rightarrow \varepsilon \Rightarrow Boolean$$

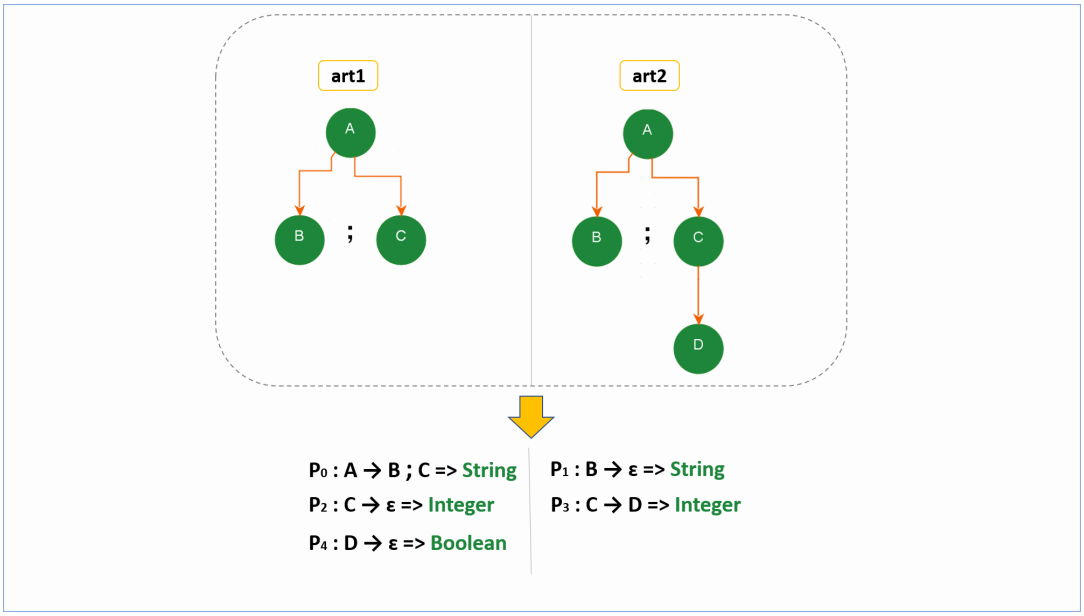


FIGURE 19 – *Modèle grammatical de workflow du processus de réclamation en utilisant le langage LSAWfP4WEB*

Dans cette section nous avons complètement modélisé le processus de réclamation à l'aide du langage *LSAWfP4WEB*. Nous avons montré comment le langage prend en compte les concepts du web tels que les entités, les type entités, les permissions et les groupes. Cependant nous devons être en mesure d'exporter et de partager les spécifications des modèles obtenus en vue de les passer plustard à un générateur et d'entamer ainsi la seconde phase de du cycle de vie de développement des processus au sein de notre application de base. Dans la section suivante nous proposons un DSEL pour l'export, la sauvegarde et le partage des spécifications du langage *LSAWfP4WEB*.

III.4. Méta-modèle du langage *LSAWfP4WEB*

Dans cette section nous présentons le méta-modèle de *LSAWfP4WEB*. Il s'agit d'un diagramme de classes permettant de construire le DSEL du langage.

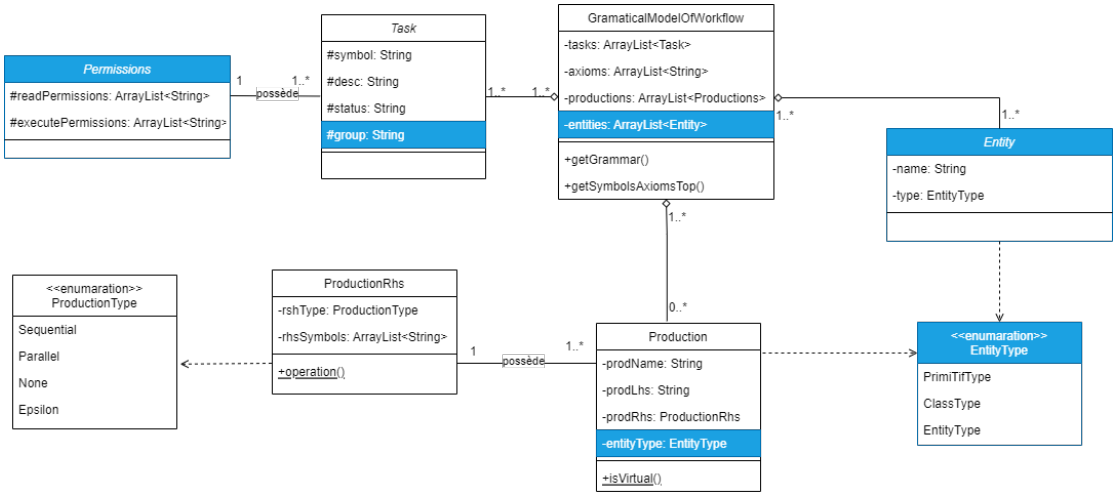


FIGURE 20 – *Méta-modèle du langage LSAWfP4WEB proposé*

Ce DSEL contient uniquement l'ensemble des éléments requis pour une spécification *LSAWfP4WEB*, et il les matérialise à l'aide des types offerts par le TypeScript. Avec ce dernier, l'on dispose d'une syntaxe claire et précise, nous fournissant tous les éléments nécessaires pour spécifier un processus avec *LSAWfP4WEB*.

III.5. Conclusion

Dans ce chapitre, nous avons proposé dans un premier une architecture d'application web permettant de faire à la fois de la gestion des processus métier mais aussi du développement classique de fonctionnalités au sein de la même application. Nous avons complètement décrit les composants de cette architecture et montré comment on arrive à résoudre les problèmes des outils existants cités dans la revue de la littérature. Dans un second temps, nous avons proposé le langage *LSAWfP4WEB* qui est un langage basé sur *LSAWfP* pour la modélisation des workflows destinés à être déployés au sein d'un application web. Enfin nous proposons un DSEL pour l'export, la sauvegarde et le partage des spécifications des processus fixant ainsi les bases d'une futur approche de génération des workflows du web à partir des spécifications du DSEL.

CONCLUSION GÉNÉRALE

SOMMAIRE

Bilan du travail 58
Quelques perspectives 59

Bilan du travail

Le domaine du Business Process Management fait ses preuves depuis de nombreuses années dans la modélisation, la spécification, l'automatisation et la maintenance des processus métier dans les entreprises. D'autre part, l'industrie du développement web ne cesse d'évoluer et propose tous les jours de nouvelles voies pour la conception d'applications web et l'automatisation d'un certain nombre de tâches au sein de celles-ci. Ainsi l'intégration de la technologie BPM dans les applications web conduit à des gains d'efficacités en réduisant le temps de création et en automatisant les processus complexes. Dans ce document, nous avons traité le problème de la gestion des processus métier dans les applications web le but étant de converger vers une approche visant à concilier le processus de développement classique avec le processus de développement des workflows au sein de la même application. Pour mener à bien ce travail, nous nous sommes fixés les objectifs suivants :

— Proposer une architecture d'application web permettant de faire à la fois de la gestion des processus métier mais aussi du développement classique de fonctionnalités au sein de la même application ;
— Proposer une extension du langage LSAWfP pour spécifier les workflows exécutables dans une application web ;
— Proposer un DSEL basé sur JSON pour le partage des spécifications des processus à exécuter dans les applications web.

Rendus au terme de ce mémoire, nous apportons les contributions suivantes :

— Une architecture d'application web permettant de faire à la fois de la gestion des processus métier mais aussi du développement classique de fonctionnalités.
— Un langage de workflows (nommé *LSAWfP4WEB*) basé sur le langage LSAWfP pour la spécification des processus à exécuter dans une application web ;
— Une extension du DSEL JSON de LSAWfP pour le langage *LSAWfP4WEB* dans le but d'exporter, de sauvegarder et de partage les spécifications des workflows du web ;
— Un prototype en cours de développement en guise de preuve de concepts.

Quelques perspectives

Dans le but de raffiner d'avantage notre approche qui pour l'instant ne possède que quelques briques de base, nous ouvrons le champs vers plusieurs pistes de recherche notamment :

— Valider l'architecture de développement web orienté processus métiers que nous proposons ;
— Améliorer l'architecture proposée pour permettre son utilisation sur une large gamme de framework d'applications web ;
— Améliorer le langage *LSAWfP4WEB* pour qu'il puisse prendre en compte les processus du web plus complexes ;
— Enrichir le langage *LSAWfP4WEB* pour qu'il prenne davantage en compte les aspects de présentation (GUI) dans les applications web ;
— Developper un éditeur du langage *LSAWfP4WEB* dans le but de faciliter la modélisation des processus ;
— Proposer des méthodes basées sur l'Ingénierie Dirigée par les Modèles pour la génération du code des processus à partir des spécifications du langage LSAWfP4WEB ;
— Définir un protocole d'exécution des processus au sein de l'application web ;
— Construire un framework permettant l'exécution des processus métier dans une application web d'une part et la création de fonctionnalités plus simples en utilisant une approche classique d'autre part.

BIBLIOGRAPHIE

[1] Dumas M, La Rosa M, Mendling J, Reijers HA, et al. Fundamentals of business process management. vol. 1. Springer ; 2013.

[2] Iskandar TF, Lubis M, Kusumasari TF, Lubis AR. Comparison between client-side and server-side rendering in the web development. In : IOP Conference Series : Materials Science and Engineering. vol. 801. IOP Publishing ; 2020. p. 012136.

[3] Yuan E, Tong J. Attributed based access control (ABAC) for web services. In : IEEE International Conference on Web Services (ICWS'05). IEEE ; 2005. .

[4] Park JS, Sandhu R, Ahn GJ. Role-based access control on the web. ACM Transactions on Information and System Security (TISSEC). 2001 ;4(1) :37-71.

[5] Schwinger W, Retschitzegger W, Schauerhuber A, Kappel G, Wimmer M, Pröll B, et al. A survey on web modeling approaches for ubiquitous web applications. International Journal of Web Information Systems. 2008.

[6] Afzal A, Akhtar A, Ishfaq N, Shafiq B, Shamail S, Elahraf A, et al. Poster : ASSEMBLE : A Collaborative Business Process Development Tool. In : 2018 IEEE/ACM 40th International Conference on Software Engineering : Companion (ICSE-Companion). IEEE ; 2018. p. 376-7.

[7] Ndadji MMZ, Tchendji MT, Djamegni CT, Parigot D. A language and methodology based on scenarios, grammars and views, for administrative business processes modelling. arXiv preprint arXiv :201013347. 2020.

[8] Van der Aalst WM. Business process management : a comprehensive survey. International Scholarly Research Notices. 2013 ;2013.

[9] Ndadji MMZ. A Grammatical Approach for Distributed Business Process Management using Structured and Cooperatively Edited Mobile Artifacts. arXiv preprint arXiv :210600041. 2021.

[10] Georgakopoulos D, Hornick M, Sheth A. An overview of workflow management : From process modeling to workflow automation infrastructure. Distributed and parallel Databases. 1995 ;3(2) :119-53.

[11] Van der Aalst WM. The application of Petri nets to workflow management. Journal of circuits, systems, and computers. 1998 ;8(01) :21-66.

[12] Van Der Aalst W, La Rosa M, Santoro F. Business process management : Donť forget to improve the process ! Business and Information Systems Engineering, 58 (1), 1–6 ; 2016.

[13] Divitini M, Hanachi C, Sibertin-Blanc C. InterOrganizational Workflows for Enterprise Coordination. In : Coordination of Internet agents. Springer ; 2001. p. 369-98.

[14] Model BP. Notation (BPMN) version 2.0. OMG Specification, Object Management Group. 2011 :22-31.

[15] White SA. Introduction to BPMN. Ibm Cooperation. 2004 ;2(0) :0.

[16] Andrews T, Curbera F, Dholakia H, Goland Y, Klein J, Leymann F, et al.. Business process execution language for web services. version ; 2003.

[17] Dumas M, La Rosa M, Mendling J, Reijers HA. In : Introduction to Business Process Management. Berlin, Heidelberg : Springer Berlin Heidelberg ; 2018. p. 1-33. Available from : https://doi.org/10.1007/978-3-662-56509-4_1.

[18] Van Der Aalst WM, Ter Hofstede AH. YAWL : yet another workflow language. Information systems. 2005 ;30(4) :245-75.

[19] Dadam P, Reichert M. The ADEPT project : a decade of research and development for robust and flexible process support. Computer Science-Research and Development. 2009 ;23(2) :81-97.

[20] Badouel E, Hélouët L, Kouamou GE, Morvan C, Fondze Jr NR. Active workspaces : distributed collaborative systems based on guarded attribute grammars. ACM SIGAPP Applied Computing Review. 2015 ;15(3) :6-34.

[21] Hollingsworth D. WfMC Standards : The Workflow Reference Model, Version 1.1. Document Number WFMC-TC-1003 Workflow Management Coalition : Winchester, England. 1995.

[22] Fakas GJ, Karakostas B. A peer to peer (P2P) architecture for dynamic workflow management. Information and Software Technology. 2004 ;46(6) :423-31.

[23] Meilin S, Guangxin Y, Yong X, Shangguang W. Workflow management systems : A survey. In : ICCT'98. 1998 International Conference on Communication Technology. Proceedings (IEEE Cat. No. 98EX243). vol. 2. IEEE ; 1998. p. 6-pp.

[24] Marchetti A, Tesconi M, Minutoli S. Xflow : An xml-based document-centric workflow. In : International Conference on Web Information Systems Engineering. Springer ; 2005. p. 290-303.

[25] Badouel E, Tchendji MT. Merging hierarchically-structured documents in workflow systems. Electronic Notes in Theoretical Computer Science. 2008 ;203(5) :3-24.

[26] Krishnan R, Munaga L, Karlapalem K. XDoC-WFMS : A framework for document centric workflow management system. In : International Conference on Conceptual Modeling. Springer ; 2001. p. 348-62.

[27] Medina-Mora R, Winograd T, Flores R, Flores F. The action workflow approach to workflow management technology. The Information Society. 1993 ;9(4) :391-404.

[28] Miao Y, Zhou J, Tao L. Realization of Key Technologies for Enterprise Collaboration Office Automation System Based on Workflow. Computer Technology and Development. 2011 ;21(3) :90-3.

[29] Damaggio E, Hull R, Vaculín R. On the equivalence of incremental and fixpoint semantics for business artifacts with Guard–Stage–Milestone lifecycles. Information Systems. 2013 ;38(4) :561-84.

[30] Hull R, Narendra NC, Nigam A. Facilitating workflow interoperation using artifact-centric hubs. In : Service-oriented computing. Springer ; 2009. p. 1-18.

[31] Lohmann N, Wolf K. Artifact-centric choreographies. In : International conference on service-oriented computing. Springer ; 2010. p. 32-46.

[32] Nigam A, Caswell NS. Business artifacts : An approach to operational specification. IBM Systems Journal. 2003 ;42(3) :428-45.

[33] Chang C, Srirama SN, Buyya R. Mobile cloud business process management system for the internet of things : a survey. ACM Computing Surveys (CSUR). 2016 ;49(4) :1-42.

[34] Tim BL. The World Wide Web : A very short personal history ; 1998. Available from : https://www.w3.org/People/Berners-Lee/ShortHistory.html.

[35] Patel K. Incremental journey for World Wide Web : introduced with Web 1.0 to recent Web 5.0–a survey paper. International Journal of Advanced Research in Computer Science and Software Engineering. 2013 ;3(10).

[36] Gellersen HW, Gaedke M. Object-oriented web application development. IEEE Internet Computing. 1999 ;3(1) :60-8.

[37] Jadhav MA, Sawant BR, Deshmukh A. Single page application using angularjs. International Journal of Computer Science and Information Technologies. 2015 ;6(3) :2876-9.

[38] Leff A, Rayfield JT. Web-application development using the model/view/controller design pattern. In : Proceedings fifth ieee international enterprise distributed object computing conference. IEEE ; 2001. p. 118-27.

[39] Fielding R, Gettys J, Mogul J, Frystyk H, Masinter L, Leach P, et al.. Hypertext transfer protocol–HTTP/1.1. RFC 2616, june ; 1999.

[40] Feng X, Shen J, Fan Y. REST : An alternative to RPC for Web services architecture. In : 2009 First International Conference on Future Information Networks ; 2009. p. 7-10.

[41] Sandhu RS, Coyne EJ, Feinstein HL, Youman CE. Role-based access control models. Computer. 1996 ;29(2) :38-47.

[42] Favre JM, Estublier J, Blay-Fornarino M. L'ingénierie dirigée par les modèles : au-delà du MDA. Hermes-Lavoisier ; 2006.

[43] Aho AV, Lam MS, Sethi R, Ullman JD. Compilers : principles, techniques, & tools. Pearson Education India ; 2007.

[44] Jézéquel JM, Combemale B, Vojtisek D. Ingénierie Dirigée par les Modèles : des concepts à la pratique... Ellipses ; 2012.

[45] Favre JM. Concepts fondamentaux de l'IDM. Actes des 2èmes journées sur l'Ingénierie Dirigée. 2006 :13.

[46] Jézéquel JM, Gérard S, Baudry B. Le génie logiciel et l'IDM : une approche unificatrice par les modèles. Lavoisier, Hermes-science ; 2006.

[47] Bézivin J. In search of a basic principle for model driven engineering. Novatica Journal, Special Issue. 2004 ;5(2) :21-4.

[48] Epner M. Poor project management number-one problem of outsourced e-projects. Research Briefs, Cutter Consortium. 2000 ;7.

[49] Souissi A. Modélisation centrée sur les processus métier pour la génération complète de portails collaboratifs. Université des Sciences et Technologie de Lille-Lille I ; 2013.

[50] Schwinger W, Koch N. Modeling web applications. Web Engineering. 2006 :39-64.

[51] Isakowitz T, Stohr EA, Balasubramanian P. RMM : A methodology for structured hypermedia design. Communications of the ACM. 1995 ;38(8) :34-44.

[52] Houben GJ, Frasincar F, Barna P, Vdovjak R. Modeling user input and hypermedia dynamics in hera. In : International Conference on Web Engineering. Springer ; 2004. p. 60-73.

[53] Ceri S, Daniel F, Matera M. Extending WebML for modeling multi-channel context-aware web applications. In : Fourth International Conference on Web Information Systems Engineering Workshops, 2003. Proceedings. IEEE ; 2003. p. 225-33.

[54] Bieber M, Isakowitz T. Designing hypermedia applications. Communications of the ACM. 1995 ;38(8) :26-9.

[55] Baresi L, Garzotto F, Paolini P. Extending UML for modeling web applications. In : Proceedings of the 34th Annual Hawaii International Conference on System Sciences. IEEE ; 2001. p. 10-pp.

[56] Troyer OD, Casteleyn S, Plessers P. WSDM : Web semantics design method. In : Web engineering : Modelling and implementing web applications. Springer ; 2008. p. 303-51.

[57] De Troyer O, Leune CJ. WSDM : a user centered design method for Web sites. Computer Networks and ISDN systems. 1998 ;30(1-7) :85-94.

[58] Plessers P, Casteleyn S, Yesilada Y, De Troyer O, Stevens R, Harper S, et al. Accessibility : a web engineering approach. In : Proceedings of the 14th international conference on World Wide Web ; 2005. p. 353-62.

[59] Schwabe D, de Almeida Pontes R, Moura I. OOHDM-Web : an environment for implementation of hypermedia applications in the WWW. ACM SigWEB Newsletter. 1999 ;8(2) :18-34.

[60] de Koch NP. Software Engineering for Adaptive Hypermedia Systems-Reference Model, Modeling Techniques and Development Process. 2001.

[61] Koch N, Kraus A. The expressive power of uml-based web engineering. In : Second International Workshop on Web-oriented Software Technology (IWWOST02). vol. 16. CYTED ; 2002. p. 40-1.

[62] Fons J, Pelechano V, Pastor O, Valderas P, Torres V. Applying the Oows Model-Driven Approach for Developing Web Applications. The Internet Movie Database Case Study. In : Web Engineering : Modelling and Implementing Web Applications. Springer London ; 2008. p. 65-108. Available from : https://doi.org/10.1007/978-1-84628-923-1_5.

[63] Gómez J, Cachero C, Pastor O. On Conceptual Modeling of Device-Independent Web Applications : Towards a Web-Engineering Approach. Ieee multimedia. 2001 ;8(2) :26-39.

[64] Robles Luna E, Rossi G, Garrigós I. WebSpec : a visual language for specifying interaction and navigation requirements in web applications. Requirements Engineering. 2011 ;16(4) :297-321.

[65] Torres V, Giner P, Pelechano V. Developing BP-driven web applications through the use of MDE techniques. Software & Systems Modeling. 2012 ;11(4) :609-31.

[66] Kappel G, Pröll B, Reich S, Retschitzegger W. Web engineering. Wiley New York ; 2006.

[67] Davis J. Open source SOA. Simon and Schuster ; 2009.

[68] Tedre M. FEATURE What should be automated ? interactions. 2008 ;15(5) :47-9.

[69] Escalona MJ, Koch N. Requirements engineering for web applications–a comparative study. Journal of web Engineering. 2003 :193-212.

[70] Kahn RL, Cannell CF. The Dynamics of Interviewing : Theory, Technique, and Cases. Theory, Technique and Cases. Wiley ; 1957. Available from : https://books.google.cm/books?id=7oMuAAAAMAAJ.

[71] Jacobson I. Object Oriented Software Engineering : A Use Case Driven Approach. Addison-Wesley ; 1992. Available from : http://www.amazon.com/Object-Oriented-Software-Engineering-Approach/dp/0201544350.

[72] Díaz E, Panach JI, Rueda S, Vanderdonckt J. An empirical study of rules for mapping BPMN models to graphical user interfaces. Multimedia Tools and Applications. 2021 ;80(7) :9813-48.

[73] Brambilla M, Butti S, Fraternali P. Webratio bpm : A tool for designing and deploying business processes on the web. In : International Conference on Web Engineering. Springer ; 2010. p. 415-29.

[74] Yongchareon S, Liu C, Zhao X, Xu J. An artifact-centric approach to generating web-based business process driven user interfaces. In : International conference on web information systems engineering. Springer ; 2010. p. 419-27.

[75] Sukaviriya N, Sinha V, Ramachandra T, Mani S, Stolze M. User-centered design and business process modeling : cross road in rapid prototyping tools. In : IFIP Conference on Human-Computer Interaction. Springer ; 2007. p. 165-78.

[76] Torres V, Pelechano V. Building business process driven web applications. In : International Conference on Business Process Management. Springer ; 2006. p. 322-37.

[77] Farahbod R, Glässer U, Vajihollahi M. Specification and validation of the business process execution language for web services. In : International Workshop on Abstract State Machines. Springer ; 2004. p. 78-94.

[78] Chabanoles N, Ozil P. Bonita BPM : An open-source BPM-based application development platform to build adaptable business applications. In : International Conference on Business Process Management ; 2015. .

[79] Red ;. Available from : `https://nodered.org/`.

[80] Afzal A, Shafiq B, Shamail S, Elahraf A, Vaidya J, Adam NR. Assemble : Attribute, structure and semantics based service mapping approach for collaborative business process development. IEEE transactions on services computing. 2018 ;14(2) :371-85.